VIGNAN'S
FOUNDATION FOR SCIENCE, TECHNOLOGY & RESEARCH
(Deemed to be University) - Estd. u/s 3 of UGC Act 1956



Course Code: 24CS951

Module -2: Unit 1

Plotting and Visualization

VIGNAN'S
FOUNDATION FOR SCIENCE, TECHNOLOGY & RESEARCH
(Deemed to be University) - Estd. u/s 3 of UGC Act 1956

- **Plotting and Visualization with matplotlib:**
  - Figures and Subplots, Colors, Markers, and Line Styles, Ticks, Labels, and Legends, Annotations
  - Drawing on a Subplot,
  - Saving Plots to File,
  - matplotlib Configuration.

- **Visualization with Seaborn:**
  - Themes, Line Plots, Bar Plots, Histograms and Density Plots, Scatter or Point Plots.
  - Distribution Plot-kdeplot.
  - Categorical Plots- boxplot, violinplot.
  - Relational Plots- Heatmaps, Pair Plots, Joint Plots, Facet Grids.

Visualization with Seaborn

- **Seaborn** is a **Python data visualization library** built on top of **Matplotlib**.
- Seaborn internally **uses Matplotlib functions** but makes them **simpler to use**.
- It simplifies the process of creating **complex visualizations** by providing built-in themes and functions designed specifically for statistical data visualization.
- It can be installed using pip:

```
pip install seaborn
```

- usually imported as

```
import seaborn as sns
```

https://seaborn.pydata.org/

- It provides a high-level interface for creating Attractive, Informative, Statistical graphics
- Widely used for Exploratory Data Analysis (EDA).

Features of Seaborn:

- Default themes
- Setting custom color palettes.
- Making attractive statistical plots.
- Easily and flexibly displaying distributions.
- Visualizing information from matrices and DataFrames.

Visualization with Seaborn: Sample Datasets

- Seaborn includes a series of sample datasets.
- These datasets allow users to experiment with Seaborn features without needing external data files.
- To see the list of datasets, use `sns.get_dataset_names()`
- Datasets are loaded using the function:

```
sns.load_dataset()
```

- The datasets are already structured as **Pandas DataFrames**, which makes them compatible with most Python data analysis tools.

```
import seaborn as sns
sns.get_dataset_names()
```

List of Data Sets

```
['anagrams',
 'anscombe',
 'attention',
 'brain_networks',
 'car_crashes',
 'diamonds',
 'dots',
 'dowjones',
 'exercise',
 'flights',
 'fmri',
 'geyser',
 'glue',
 'healthexp',
 'iris',
 'mpg',
 'penguins',
 'planets',
 'seaice',
 'taxis',
 'tips',
 'titanic']
```

Visualization with Seaborn: Sample Datasets

Some commonly used datasets:

Dataset	Description
`tips`	Restaurant tipping data
`iris`	Classic dataset for classification problems
`titanic`	Passenger survival data
`flights`	Airline passenger traffic over time
`car_crashes`	Traffic accident statistics
`penguins`	Penguin species measurements

Example


```
import seaborn as sns

data = sns.load_dataset("tips")
print(data.head())
```

```
   total_bill   tip     sex smoker  day    time  size
0       16.99  1.01  Female     No  Sun  Dinner     2
1       10.34  1.66    Male     No  Sun  Dinner     3
2       21.01  3.50    Male     No  Sun  Dinner     3
3       23.68  3.31    Male     No  Sun  Dinner     2
4       24.59  3.61  Female     No  Sun  Dinner     4
```

VIGNAN'S
FOUNDATION FOR SCIENCE, TECHNOLOGY & RESEARCH
(Deemed to be University) - Estd. u/s 3 of UGC Act 1956

- Seaborn supports many types of visualizations

Plot Type	Syntax
Line Plot	`sns.lineplot()`
Bar Plot	`sns.barplot()`
Scatter Plot	`sns.scatterplot()`
Histogram	`sns.histplot()`
Box Plot	`sns.boxplot()`
Heatmap	`sns.heatmap()`

https://seaborn.pydata.org/tutorial.html

https://www.kaggle.com/code/saurav9786/seaborn-tutorial

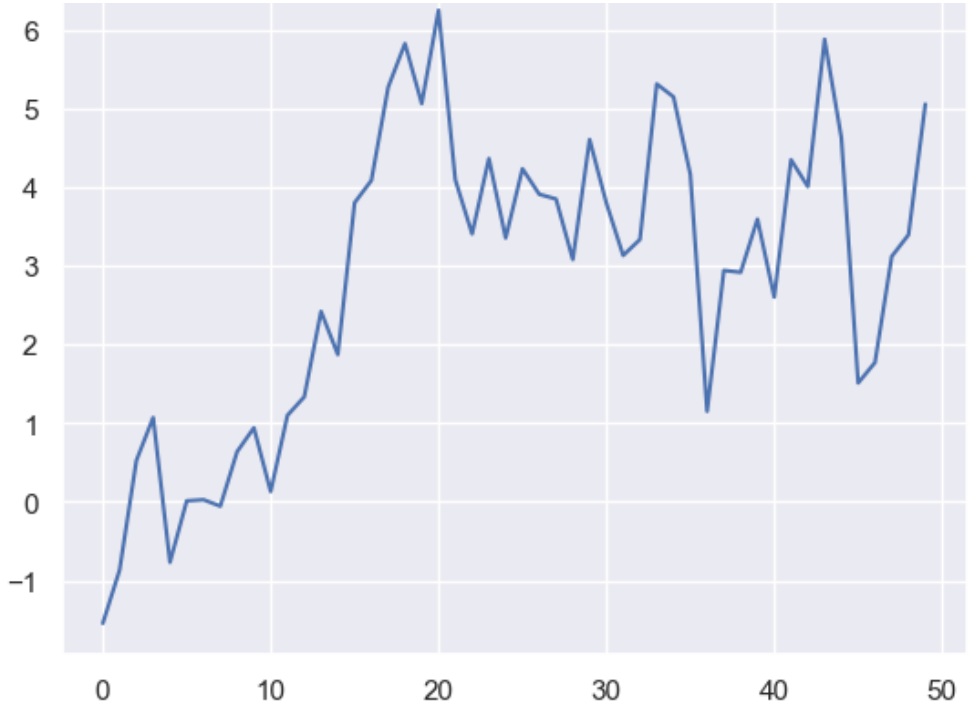
Parameter	Description
data	Dataset used for plotting
x	Variable for x-axis
y	Variable for y-axis
hue	Adds color grouping
style	Different markers for categories
size	Vary marker size

Visualization with Seaborn: Themes

- Seaborn automatically applies beautiful and readable themes to plots.
- These themes improve
  - Visual and Presentation quality
  - Data interpretation
- Common Theme styles:
  - **`darkgrid`** – default theme, and has a gray background with visible grid lines
  - **`whitegrid`** – white background with grid lines
  - **`dark`** – gray background but no grid lines
  - **`white`** – plain white background without grid lines
  - **`ticks`** – similar to the white theme but includes axis ticks (small marks on axes)

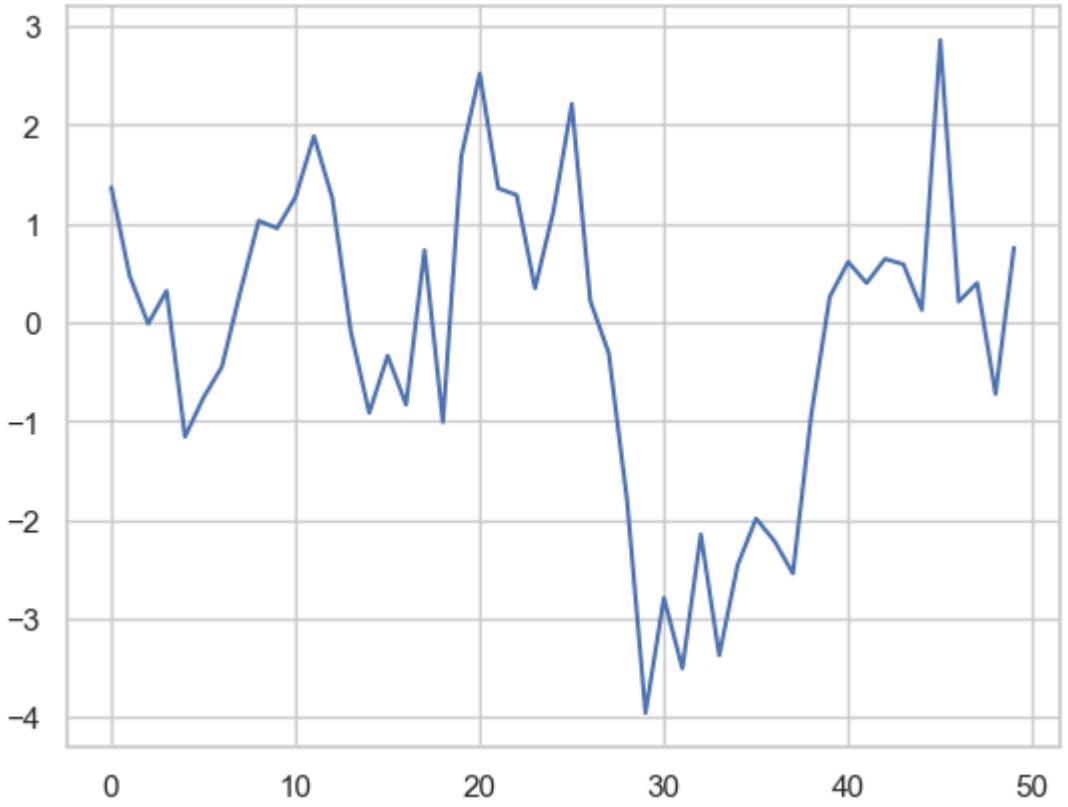
Example

```
import seaborn as sns
sns.set_theme(style="darkgrid")
sns.lineplot(np.random.standard_
normal(50).cumsum())
```

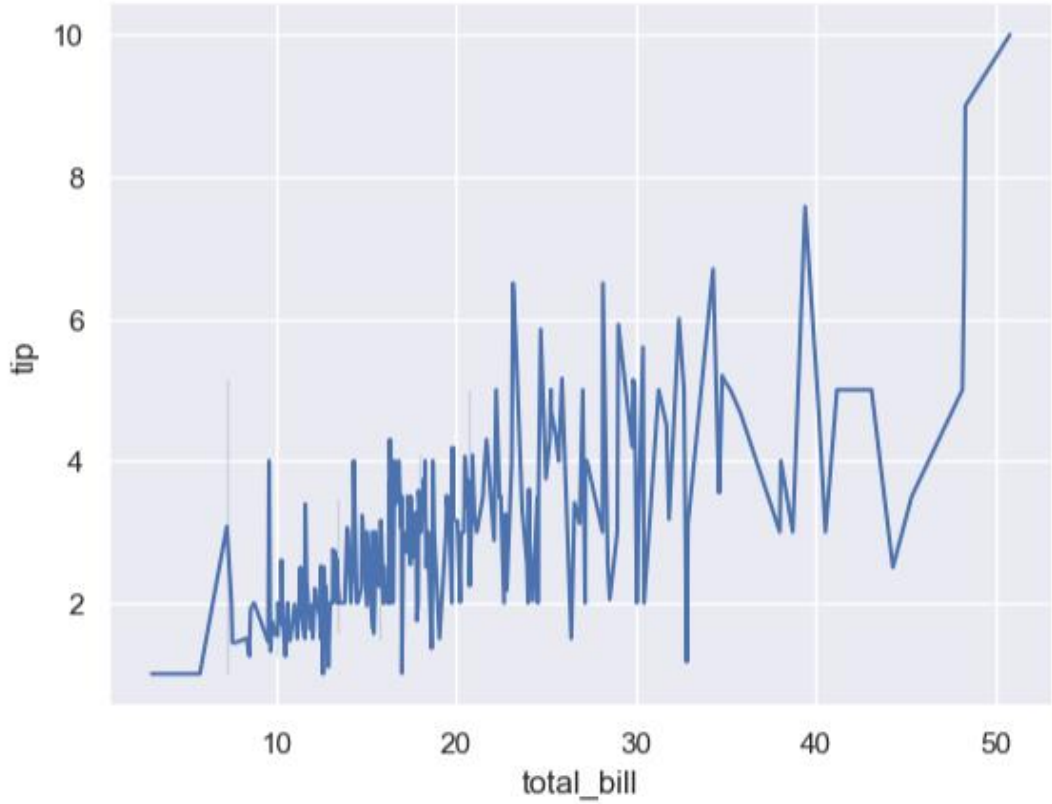


Example

```
import seaborn as sns
sns.set_theme(style="whitegrid")
sns.lineplot(np.random.standard_
normal(50).cumsum())
```




```
tips = sns.load_dataset("tips")
sns.set_theme(style="darkgrid")
sns.lineplot(data=tips,x="total_bill",
                    y="tip")
```



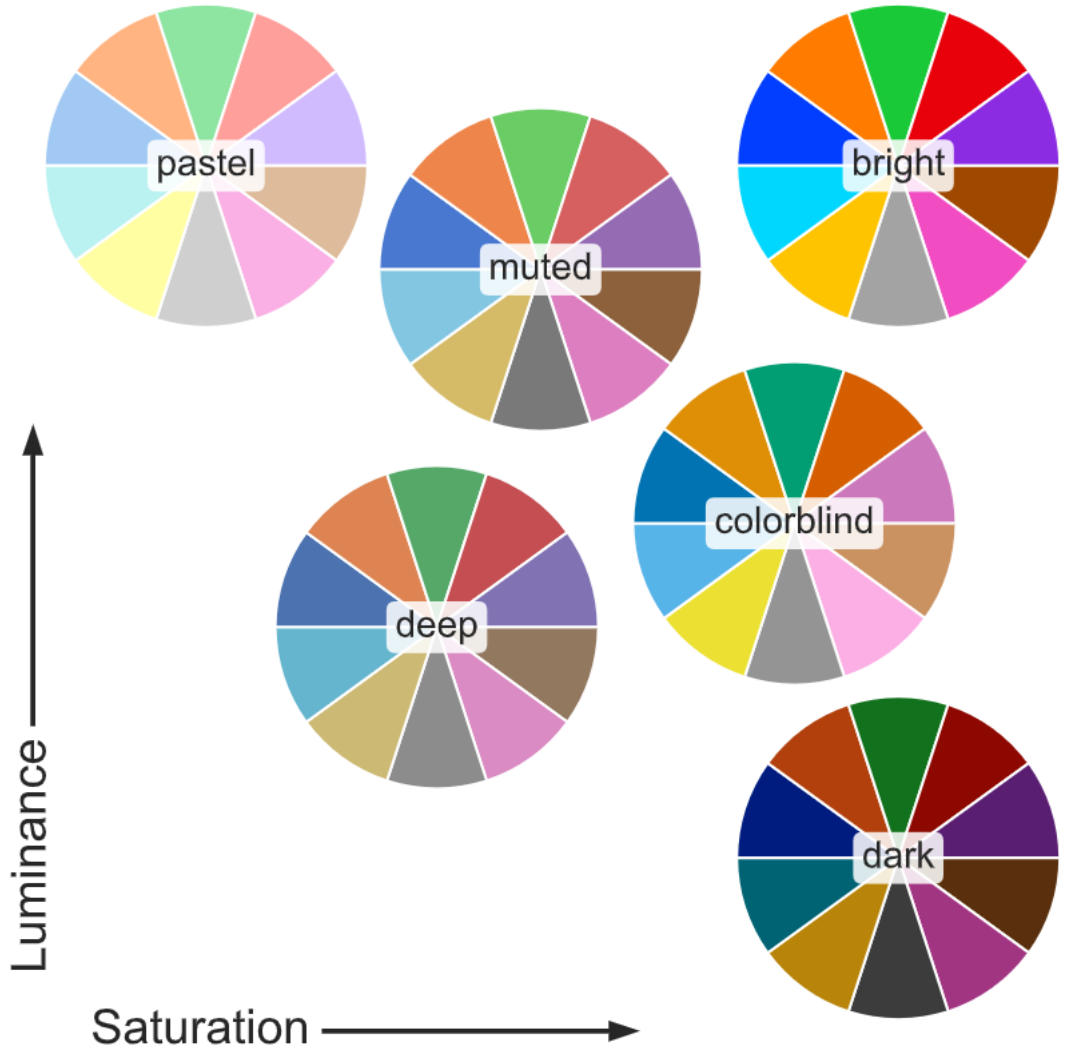
Visualization with Seaborn: custom color palettes

- It is a collection of colors used in a visualization.
- Provides many built-in color palettes that make visualizations more attractive and meaningful.
- Benfits:
  - Highlights patterns in data
  - Differentiates categories clearly
  - Improves readability
- Main types of color palettes in Seaborn:
  - Qualitative palettes
  - Sequential palettes
  - Diverging palettes

VIGNAN'S
FOUNDATION FOR SCIENCE, TECHNOLOGY & RESEARCH
(Deemed to be University) - Estd. u/s 3 of UGC Act 1956

Qualitative palettes:

- used for categorical data, where different categories must be clearly distinguished.
- Examples of categorical variables: Product types, Departments, Gender
- Common Qualitative Palettes
  - deep
  - muted
  - pastel
  - Bright
  - dark
  - colorblind

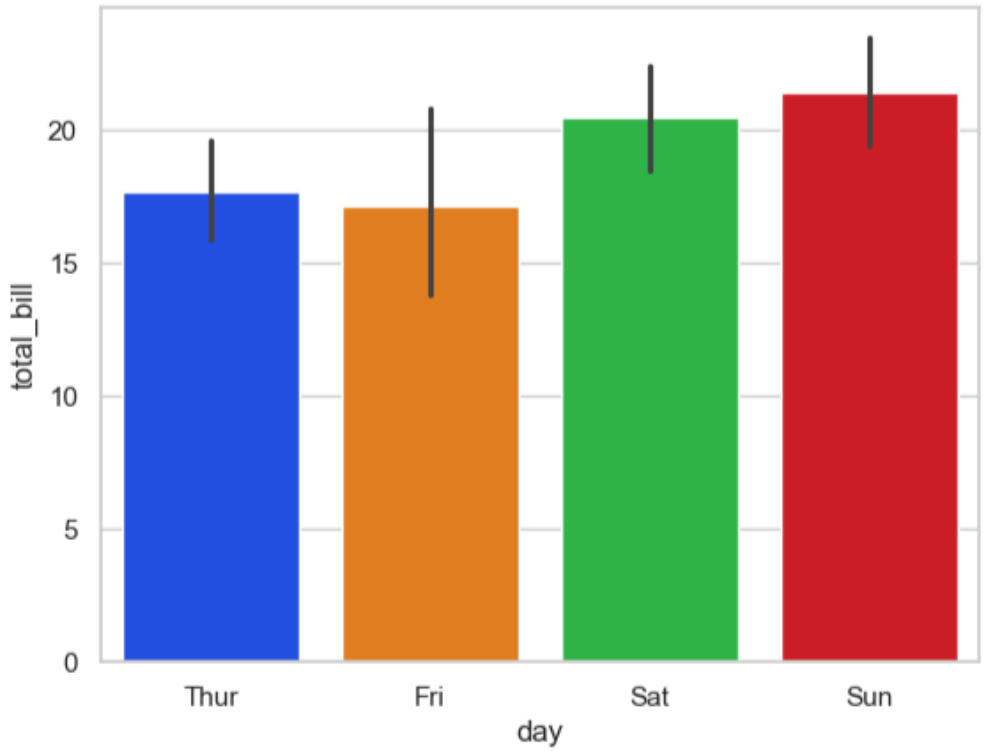


Example


```
import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset
tips = sns.load_dataset("tips")
# Set color palette
sns.set_palette("bright")
# Bar plot with categorical coloring
sns.barplot(x="day", y="total_bill", hue="day", data=tips)

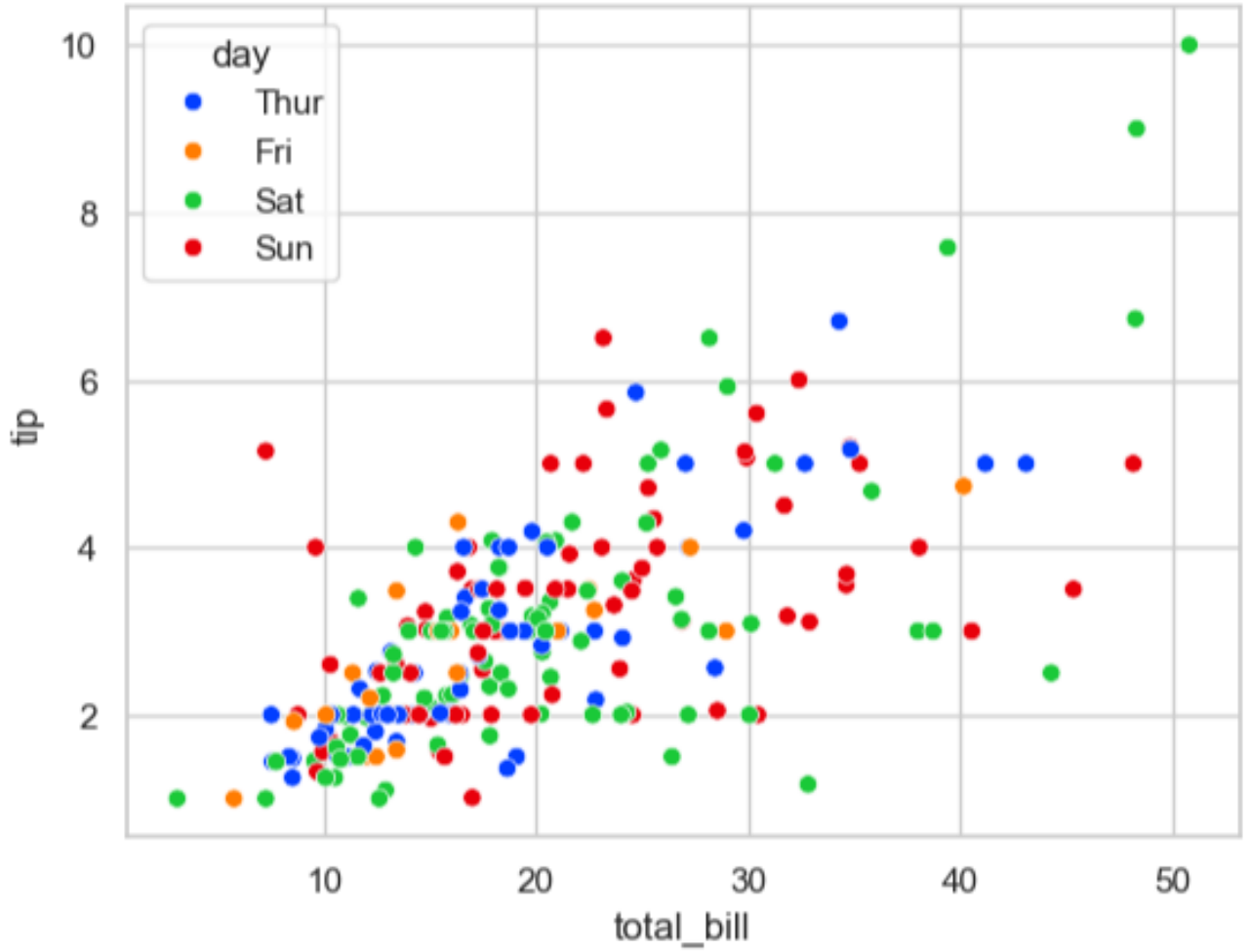
plt.show()
```



Example


```
import seaborn as sns
import matplotlib.pyplot as plt

tips = sns.load_dataset("tips")
sns.scatterplot(
    data=tips,
    x="total_bill",
    y="tip",
    hue="day",
    palette="bright"
)
plt.show()
```



VIGNAN'S
FOUNDATION FOR SCIENCE, TECHNOLOGY & RESEARCH
(Deemed to be University) - Estd. u/s 3 of UGC Act 1956

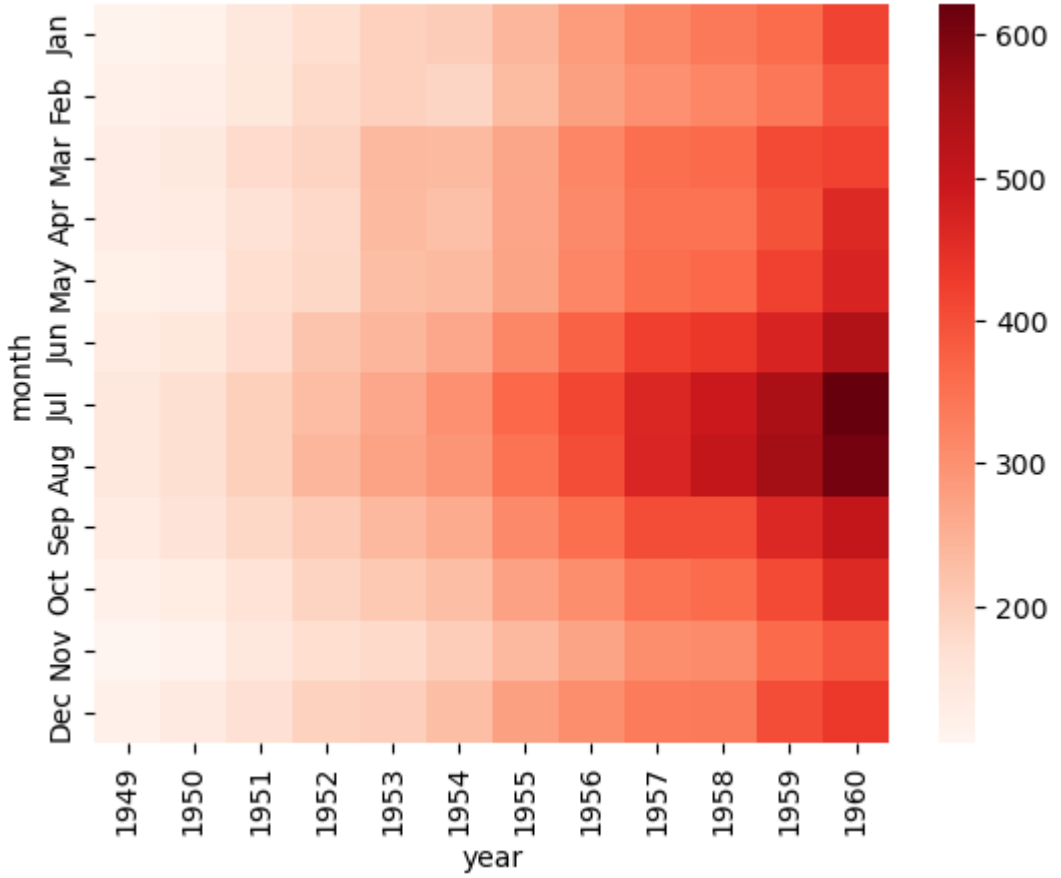
Sequential Color Palettes:

- These are used when data values progress from low to high.
- These palettes use variations of a single color where intensity represents magnitude.
  - Lighter colors represent smaller values
  - Darker colors represent larger values
- Use cases: Heatmaps, density plots, and numerical data that increases continuously.
- Common Sequential Palettes
  - Blues
  - Greens
  - Oranges
  - Purples
  - Reds

Example

```
import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset
flights = sns.load_dataset("flights")
# Convert data into matrix form
data = flights.pivot(index="month", columns="year", values="passengers")
# Plot heatmap
sns.heatmap(data, cmap="Reds")
plt.show()
```



VIGNAN'S
FOUNDATION FOR SCIENCE, TECHNOLOGY & RESEARCH
(Deemed to be University) - Estd. u/s 3 of UGC Act 1956

Diverging Color Palettes:

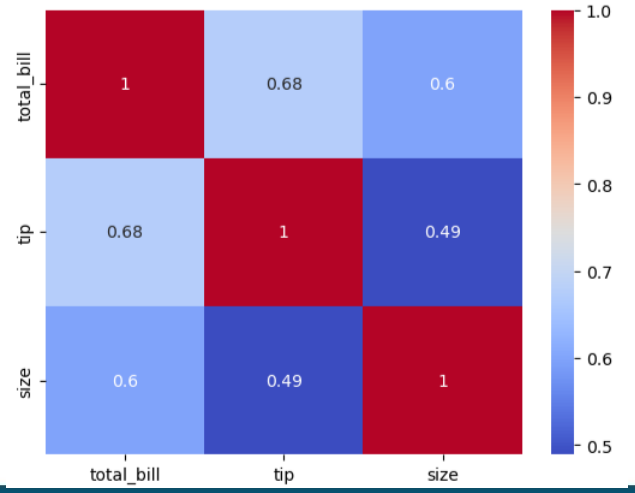
- These are used when data has a central value or midpoint, such as zero or an average.
- These palettes use two contrasting colors that diverge from a neutral midpoint.
- Use cases: temperature differences, correlation matrices, and financial gain/loss data.
- Common Diverging Palettes
  - coolwarm: color transition: **Blue → White → Red**
  - RdBu: color transition: **Red → White → Blue** with Strong contrast
  - Spectral: Transition: **Red → Orange → Yellow → Green → Blue**

Example


```
import matplotlib.pyplot as plt
import numpy as np

tips = sns.load_dataset("tips") # Loads a built-in dataset
numeric_data = tips.select_dtypes(include=['number']) # Filters only numeric columns
corr = numeric_data.corr()

plt.figure()
sns.heatmap(corr, cmap="coolwarm")  # Try "RdBu" or "Spectral"
plt.show()
```



- Blue → negative correlation
- Red → positive correlation
- White → neutral or zero

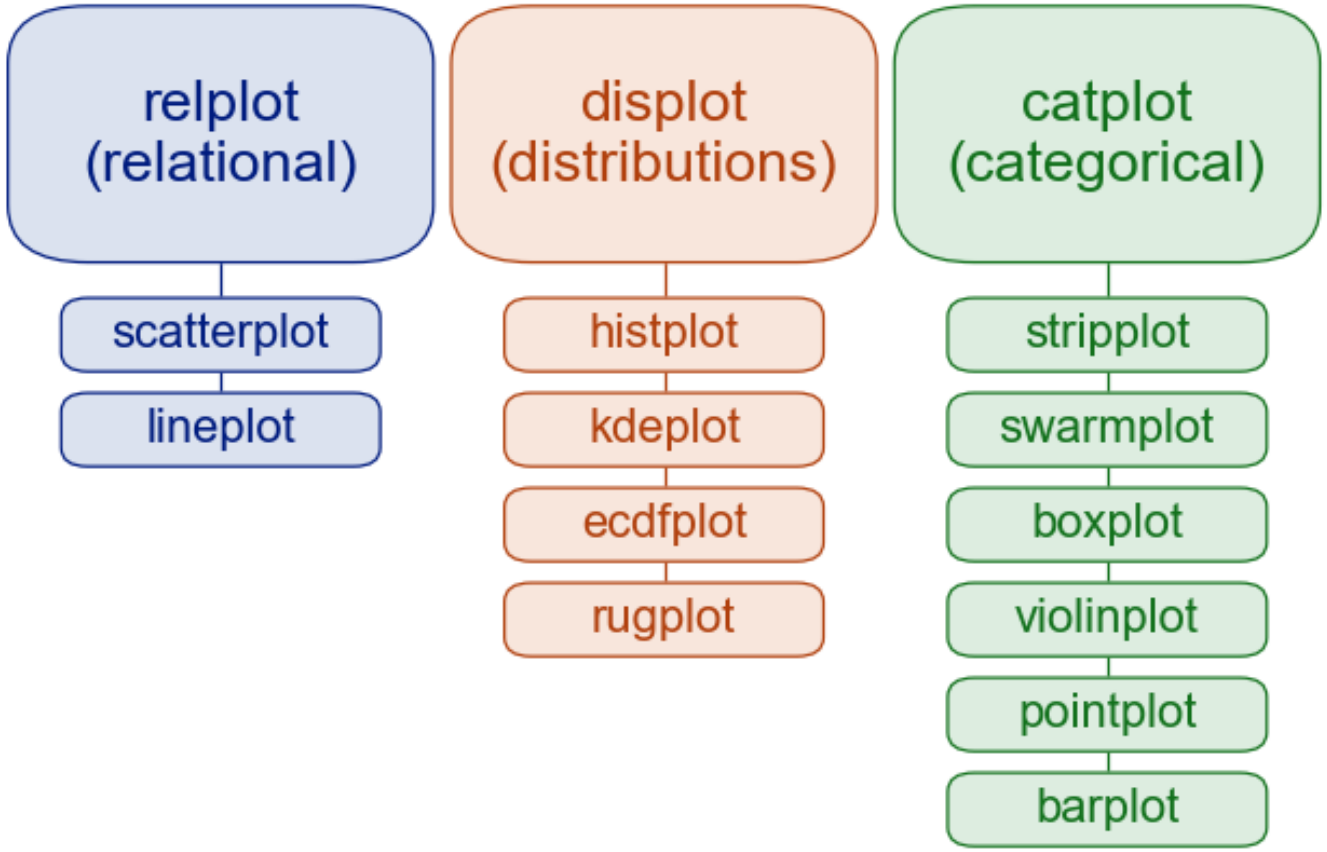
Types of plots in Seaborn

analyze relationships between variables (numerical)

show how data values are distributed

visualize relationships involving categorical variables.

compare groups or categories



Relational plots (relplot)

- Scatterplot: relationship between two numerical variables
- Lineplot: display trends over continuous data

```
sns.relplot(data=, x=" ", y=" ", kind="scatter")
```

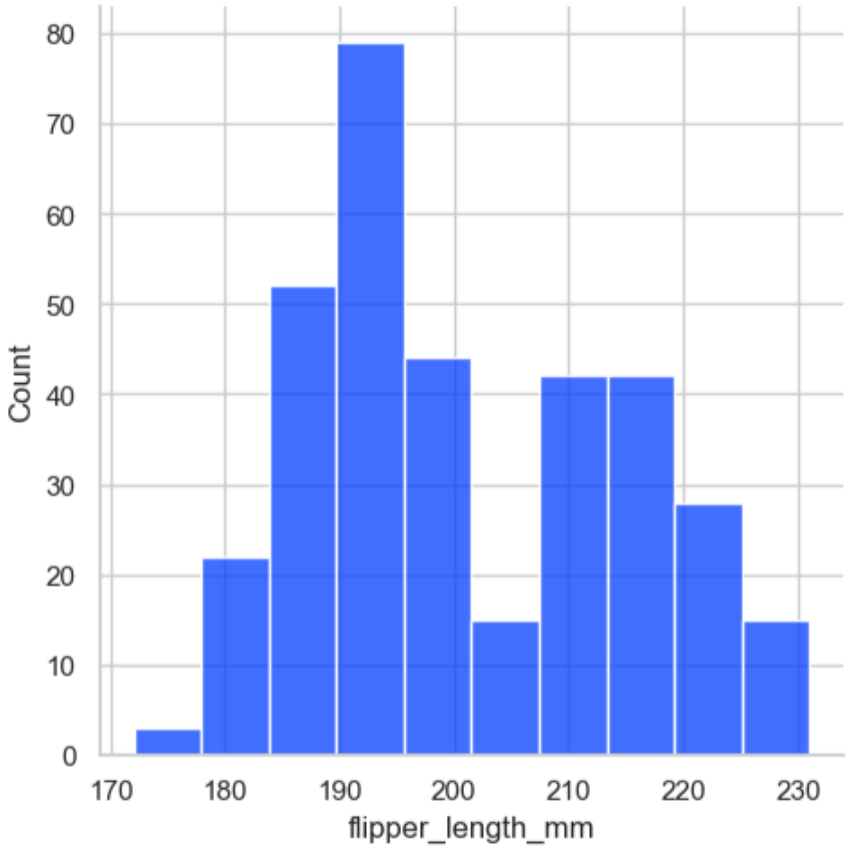
Parameter	Description
data	Dataset used for plotting
x	Variable for x-axis
y	Variable for y-axis
kind	Type of relational plot (scatter or line)
hue	Adds color grouping
style	Different markers for categories
size	Vary marker size
col	Creates multiple subplots

Distribution plots (dislplot)

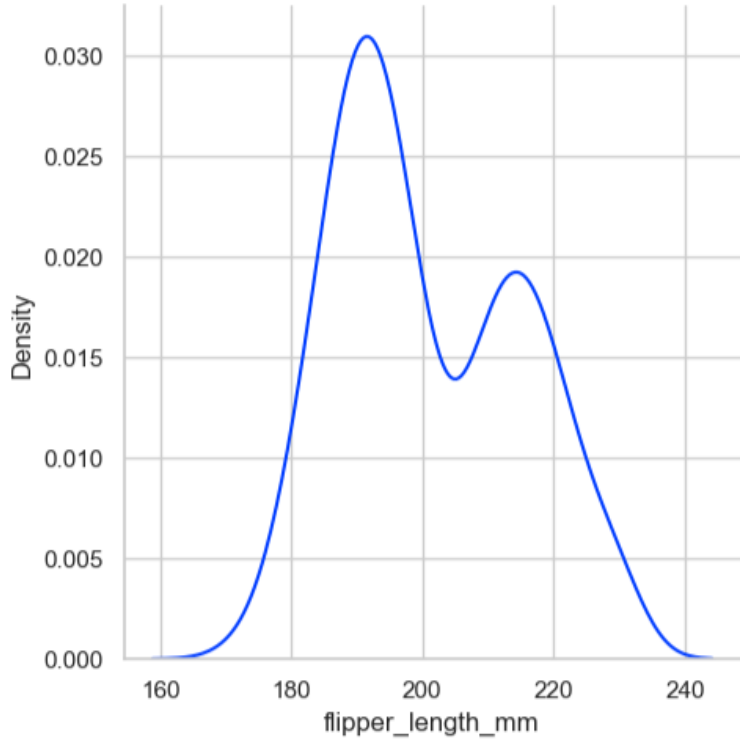
- These help understand:
  - spread of data
  - Skewness
  - probability distribution
- `displot()` is the figure-level function for the distributions module.
- Its default behavior is to draw a histogram, using the same code as `histplot()`
- `histplot()` → Shows frequency distribution of data
- `kdeplot()` → Displays a smooth probability density curve.
- `ecdfplot()` → Shows the cumulative distribution of data.
- `rugplot()` → Displays individual data points along an axis.

Example- histplot()

```
penguins = sns.load_dataset("penguins")
sns.displot(data=penguins, x="flipper_length_mm")
```

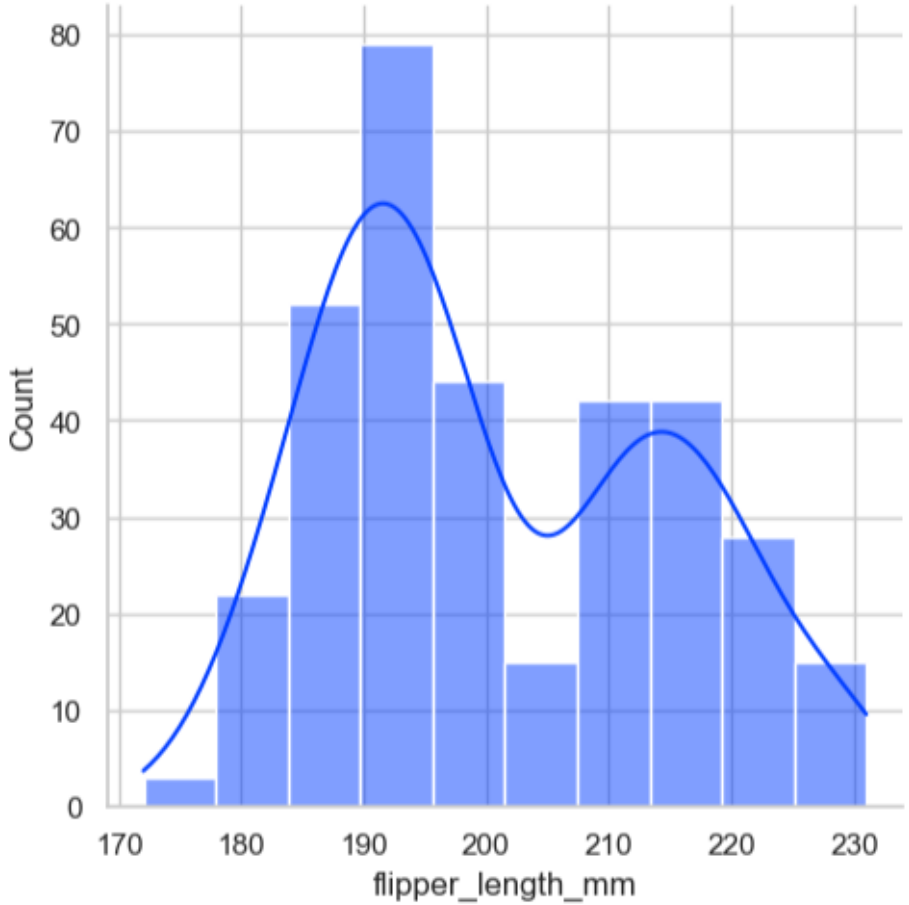



```
sns.displot(data=penguins,
x="flipper_length_mm", kind="kde")
```



Example- histplot()

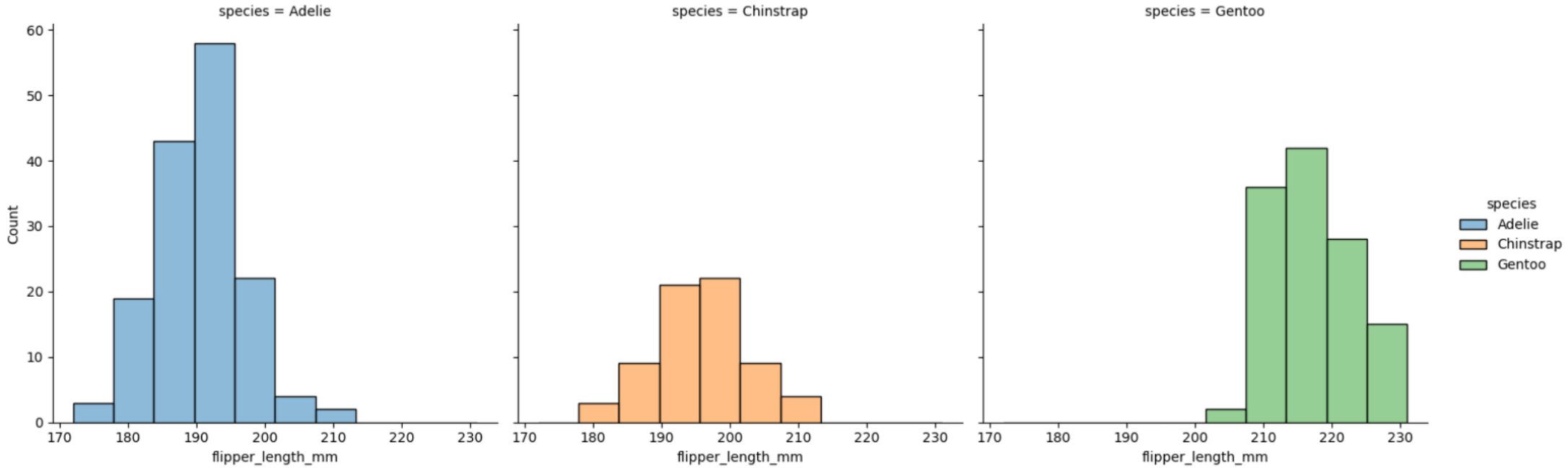

```
penguins = sns.load_dataset("penguins")
sns.displot(data=penguins, x="flipper_length_mm", kde=True)
```



Example- histplot()

- `displot()` can automatically create separate plots .

```
sns.displot(data=penguins, x="flipper_length_mm", hue="species",
col="species")
```

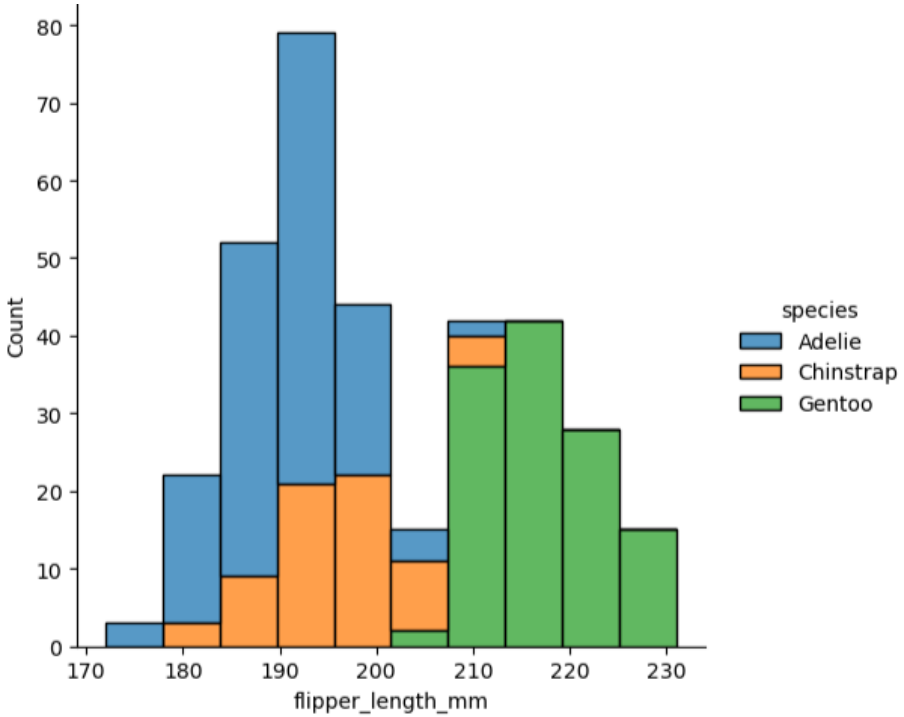


Example- histplot()


```
penguins = sns.load_dataset("penguins")
sns.displot(data=penguins, x="flipper_length_mm", hue="species", multiple="stack")
```

OR

```
sns.histplot(data=penguins, x="flipper_length_mm", hue="species", multiple="stack")
```

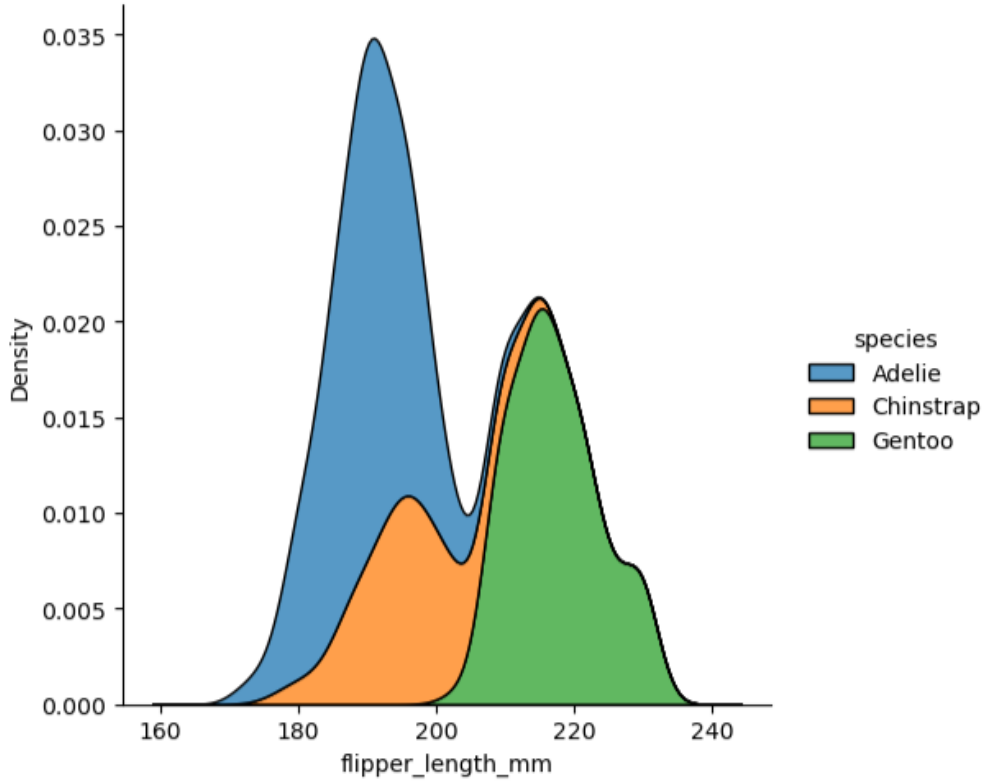


Example – kdeplot()

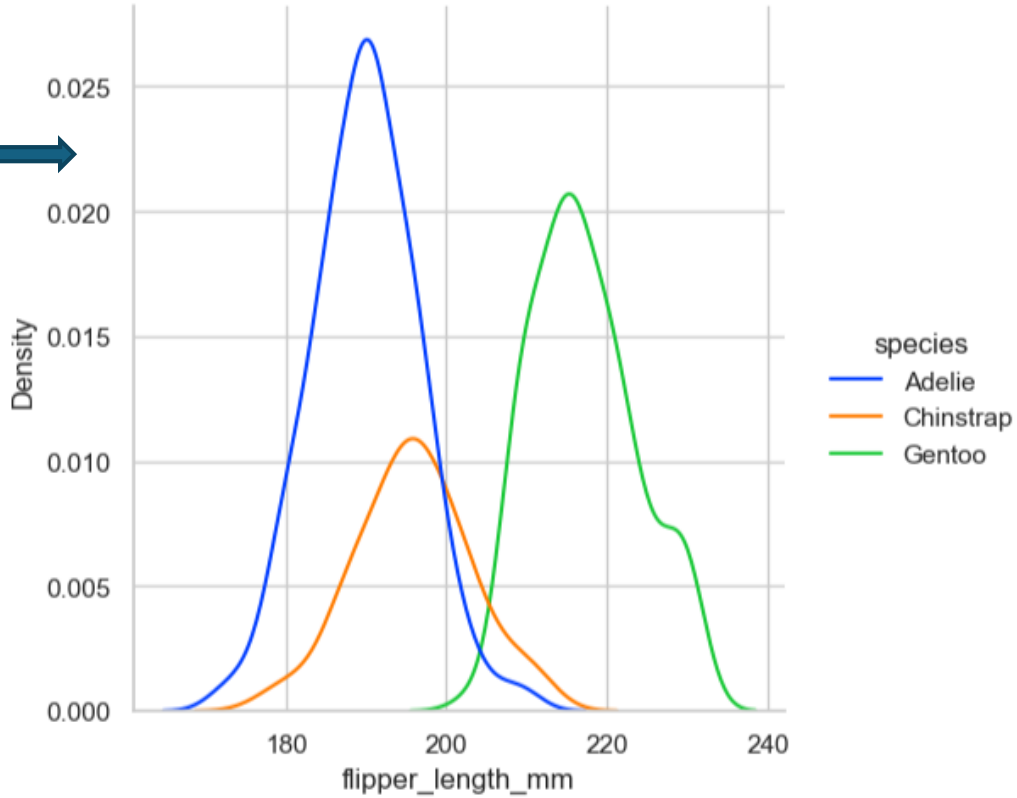
```
sns.displot(data=penguins, x="flipper_length_mm", hue="species",
multiple="stack", kind="kde")
```

OR

```
sns.kdeplot(data=penguins, x="flipper_length_mm", hue="species",
multiple="stack")
```

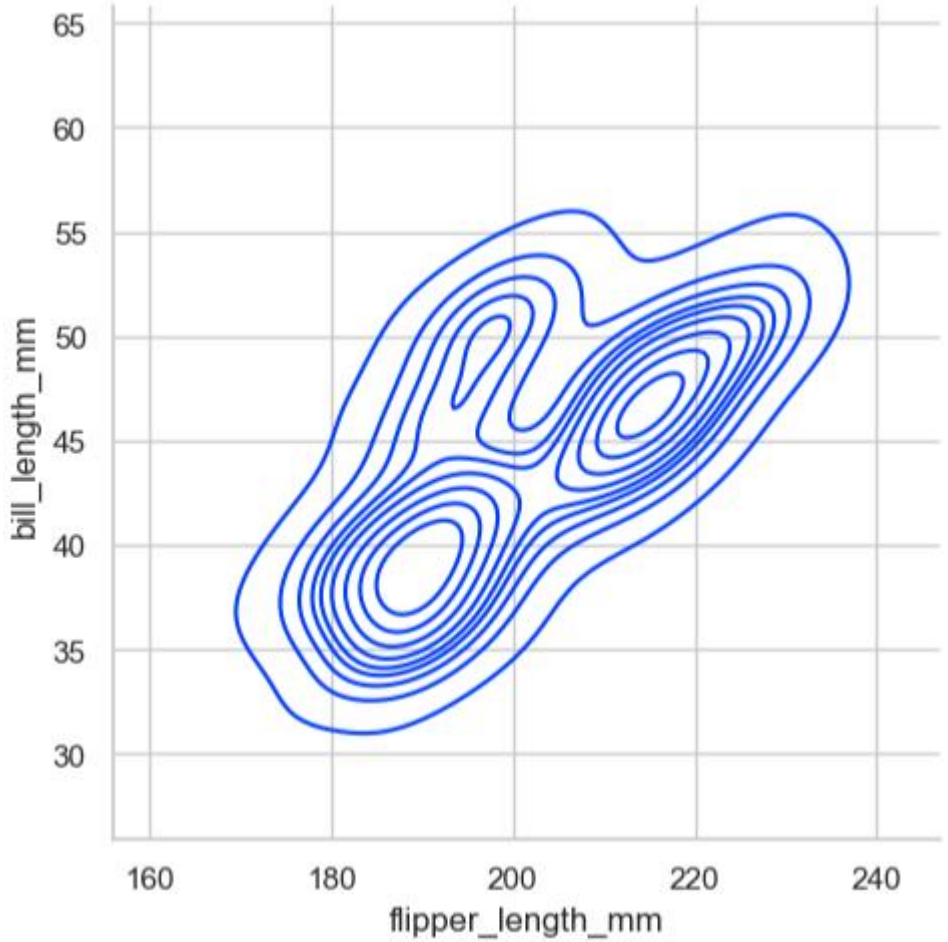


Without keyword, multiple ="stack"



Example – kdeplot()


```
sns.displot(data=penguins, x="flipper_length_mm", y="bill_length_mm",
kind="kde")
```

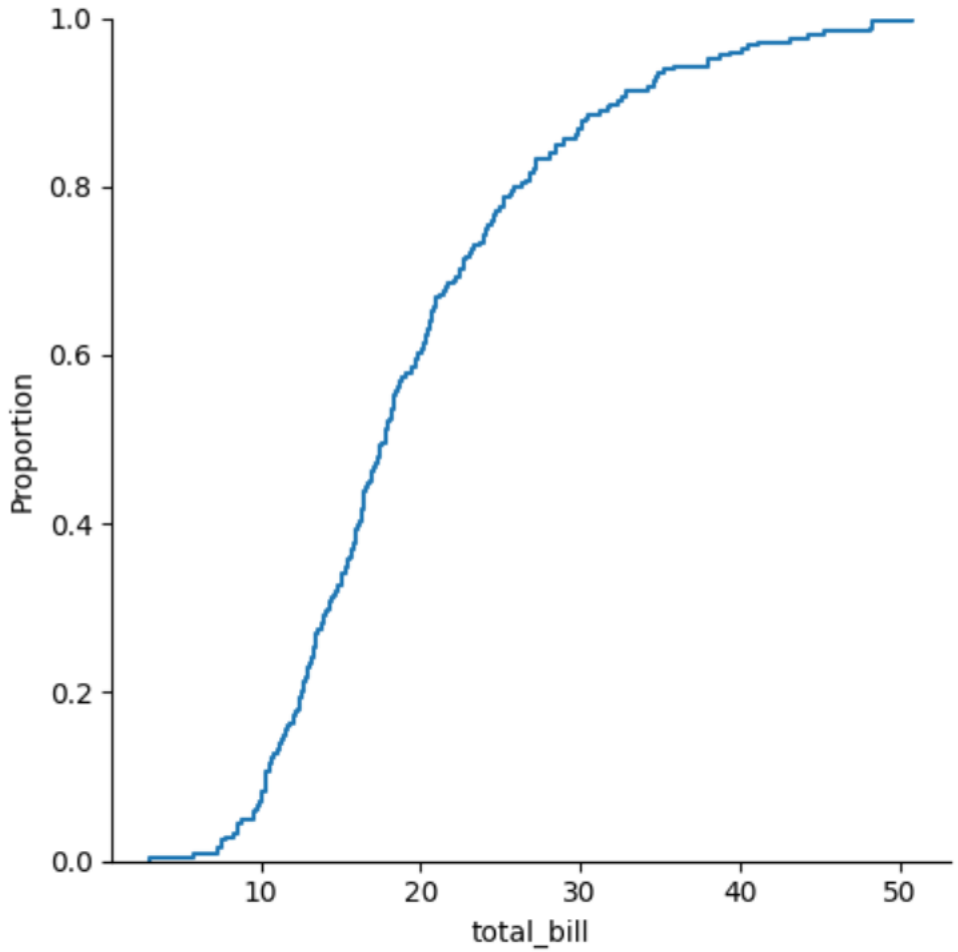


Example – ecdfplot()


```
sns.displot(data=tips, x="total_bill",kind="ecdf")
```

OR

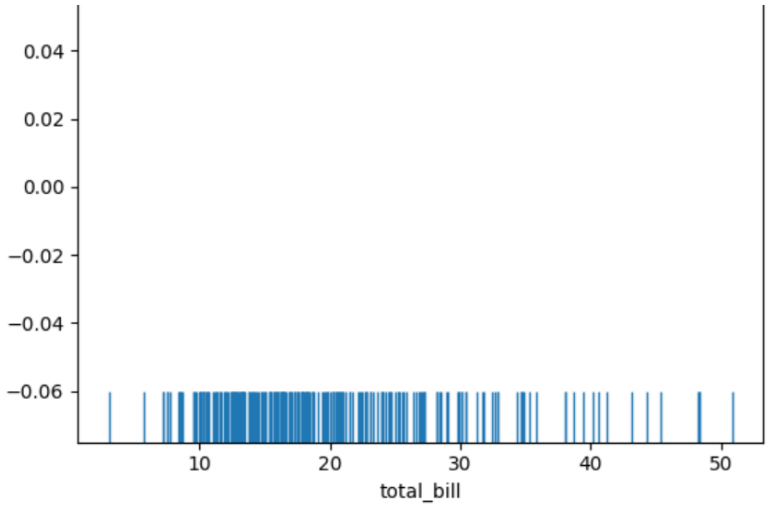
```
sns.ecdfplot(data=tips, x="total_bill")
```



rugplot()

- It draws **small tick marks (lines)** on the axis to represent each observation in the dataset.

Parameter	Description
data	Dataset used for plotting
x	Variable for horizontal axis
y	Variable for vertical axis
height	Height of rug lines
color	Color of the rug marks




```
sns.rugplot(data=tips, x="total_bill", height=0.1)
```

Categorical plots (catlplot)

- It helps compare different categories or groups in a dataset.

```
sns.catplot(data=None, x=None, y=None, kind="strip")
```

Parameter	Description
data	Dataset used for plotting
x	Categorical variable
y	Numerical variable
kind	Type of categorical plot
hue	Adds color grouping
col	Creates multiple subplots

Categorical plots (catlplot)

- supports several kind of plot types:

Categorical scatterplots:

- `stripplot()` (with `kind="strip"`; the default)
- `swarmplot()` (with `kind="swarm"`)

Categorical distribution plots:

- `boxplot()` (with `kind="box"`)
- `violinplot()` (with `kind="violin"`)
- `boxenplot()` (with `kind="boxen"`)

Categorical estimate plots:

- `pointplot()` (with `kind="point"`)
- `barplot()` (with `kind="bar"`)
- `countplot()` (with `kind="count"`)

https://seaborn.pydata.org/tutorial/categorical.html

Categorical scatter plots

- The default visualization used by **`catlplot()`** is a categorical scatter plot, specifically a strip plot **`striplplot()`**
- **Strip plot**
  - shows **individual data points for each category**.
  - To prevent overlapping points, Seaborn introduces a small **random displacement called "jitter"**.
  - **Jitter** slightly shifts points horizontally so that overlapping points become visible.
- **Swarm plot**
  - Similar to strip plot but but uses a **smarter algorithm to prevent overlapping points completely**.
  - Instead of random jitter, points are arranged **systematically so that they never overlap**.

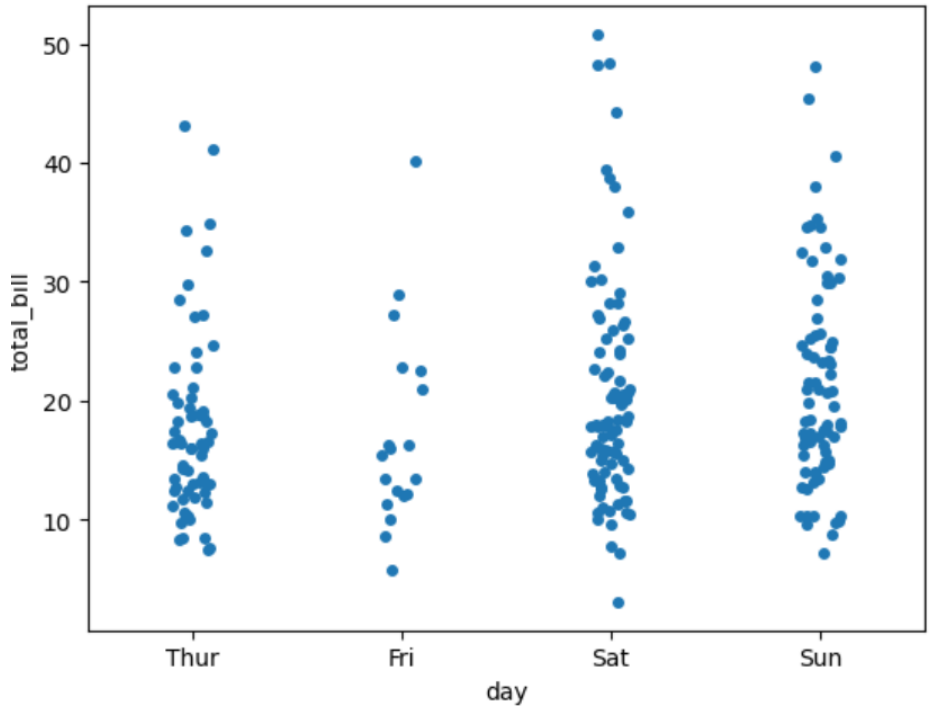
Example – stripplot()

VIGNAN'S
FOUNDATION FOR SCIENCE, TECHNOLOGY & RESEARCH
(Deemed to be University) - Estd. u/s 3 of UGC Act 1956

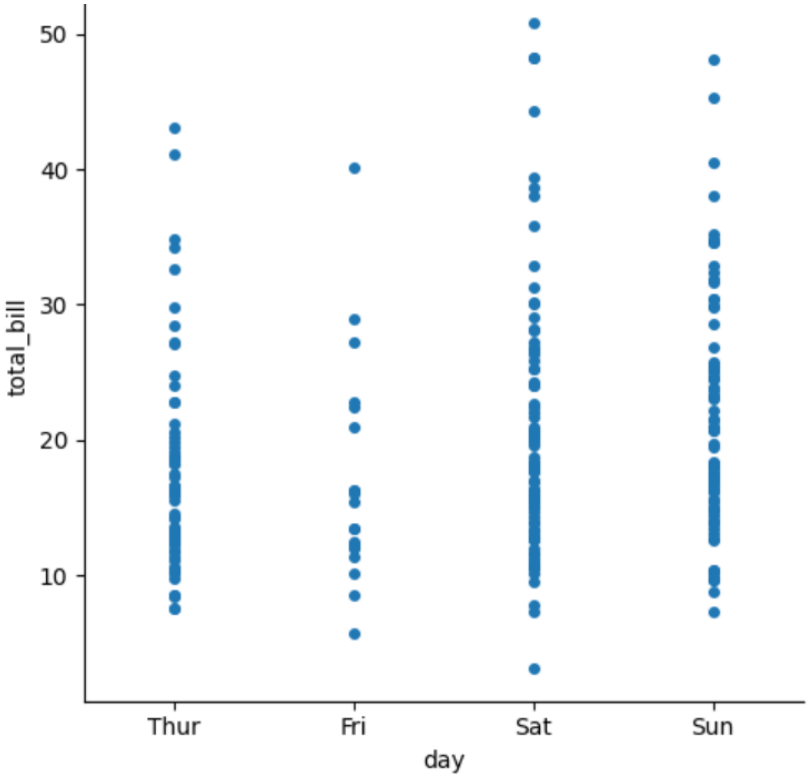
```
sns.catplot(data=tips, x="day", y="total_bill",
kind="strip")
```

OR

```
sns.stripplot(data=tips, x="day", y="total_bill")
```



For
jitter=False



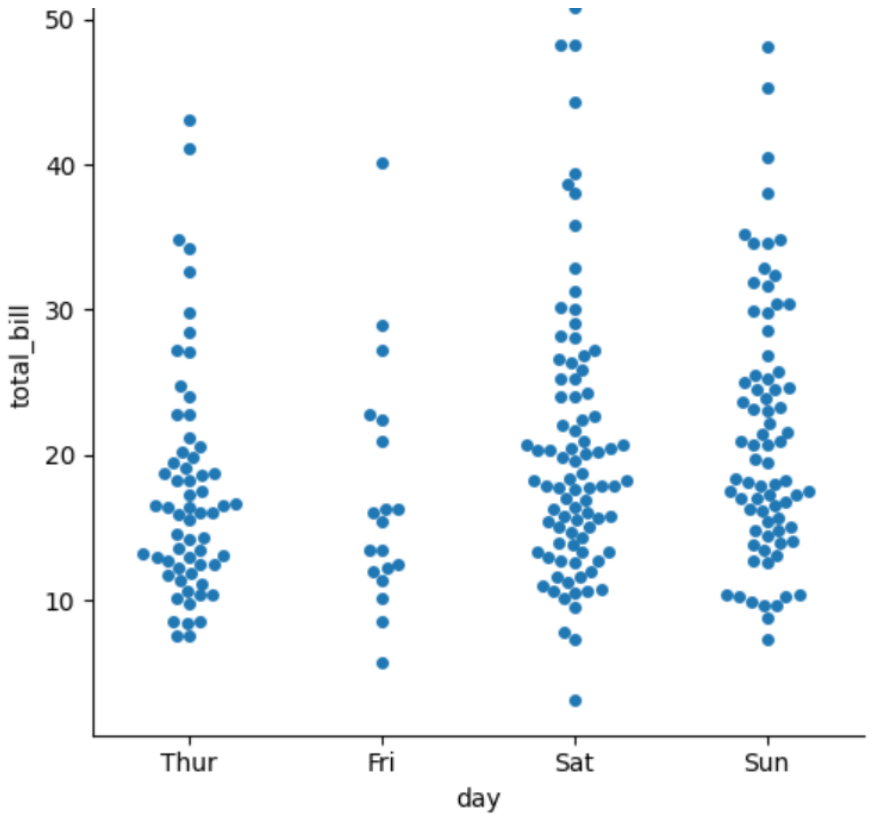
Example – swarmplot()

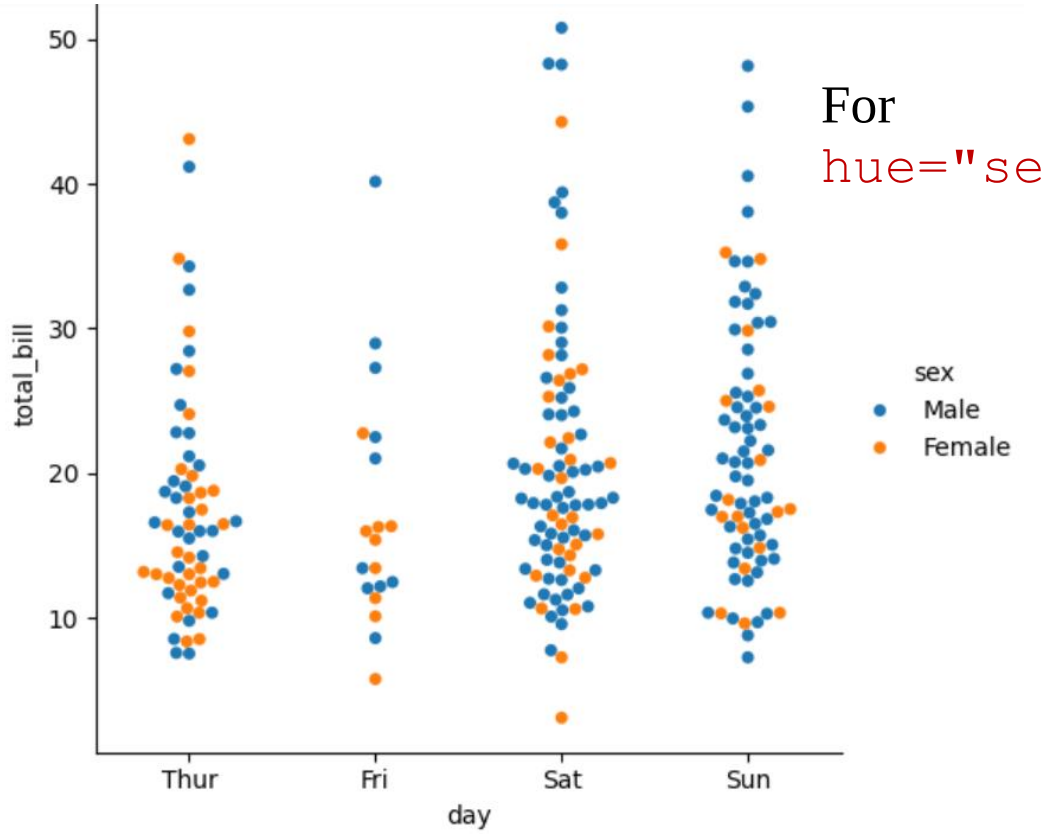
VIGNAN'S
FOUNDATION FOR SCIENCE, TECHNOLOGY & RESEARCH
(Deemed to be University) - Estd. u/s 3 of UGC Act 1956

```
sns.catplot(data=tips, x="day", y="total_bill",kind="swarm")
```

OR

```
sns.swarmplot(data=tips, x="day", y="total_bill")
```





For
`hue="sex"`

Categorical distribution plots

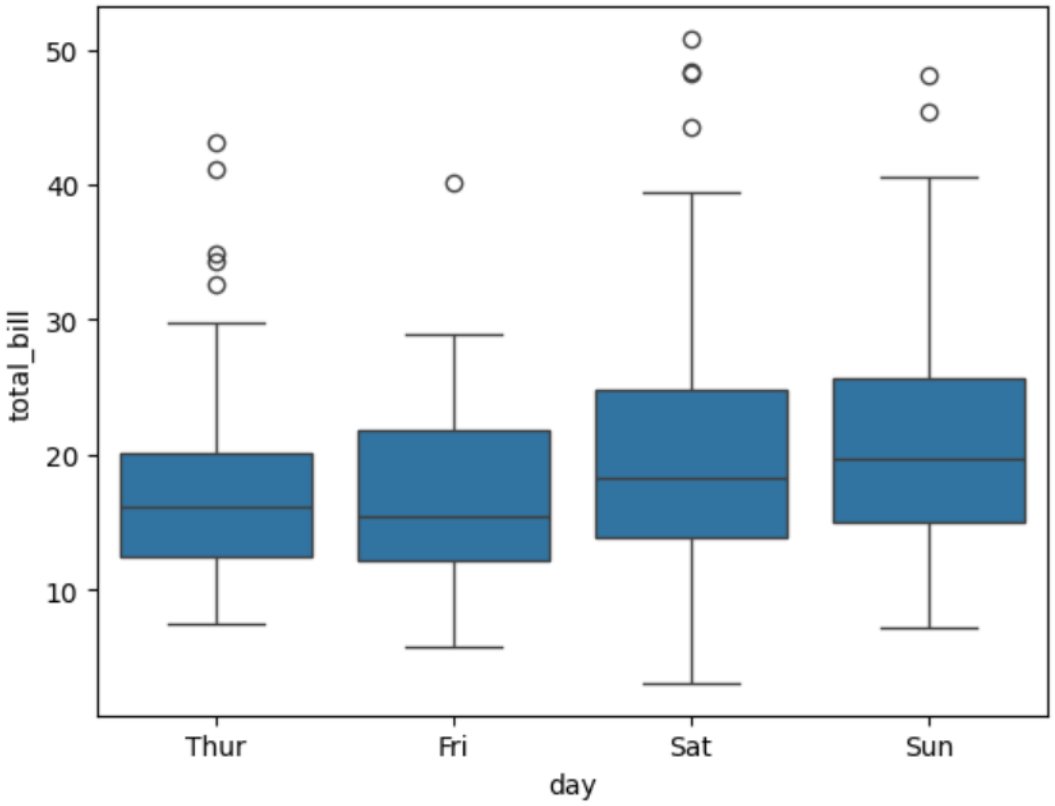
- When the **dataset size increases**, categorical scatter plots (strip and swarm plots) may become difficult to interpret because many points **overlap or crowd together**.
- To solve this problem, use **summary plots** that describe the **distribution of data within categories**.
- **Box plot**
  - statistical distribution using quartiles.
- **Boxen plot**
  - similar to a box plot but optimized for showing more information about the shape of the distribution.
  - It is best suited for larger datasets
- **Violin plot**
  - combines box plot and distribution density.

Example – boxplot()

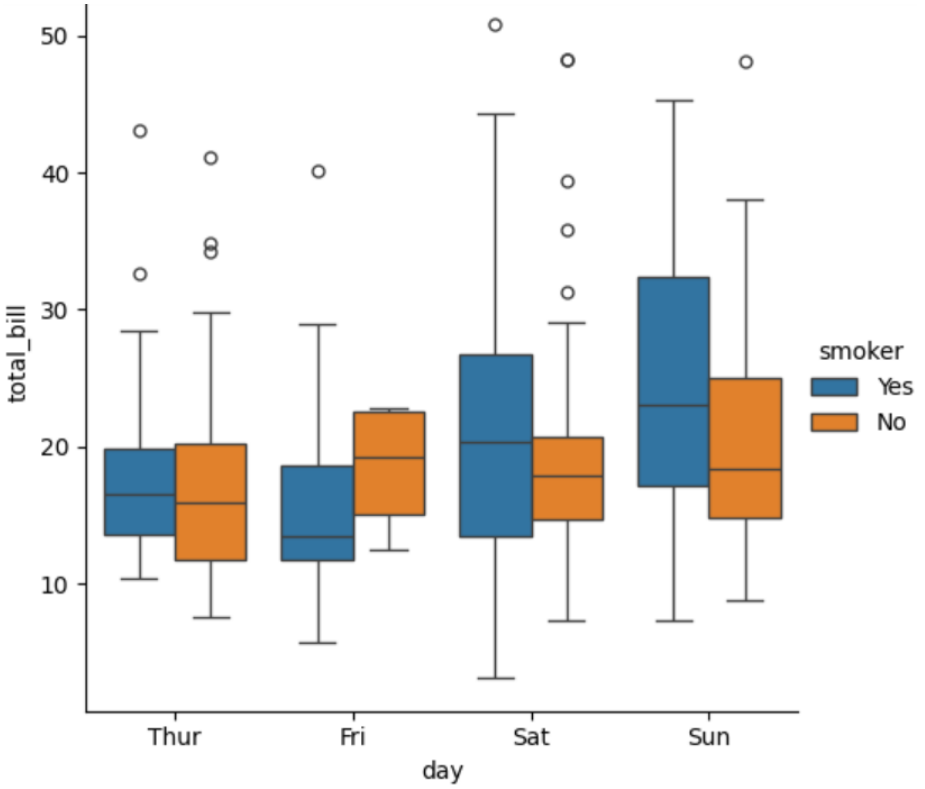

```
sns.catplot(data=tips, x="day", y="total_bill", kind="box")
```

Or

```
sns.boxplot(data=tips, x="day", y="total_bill")
```



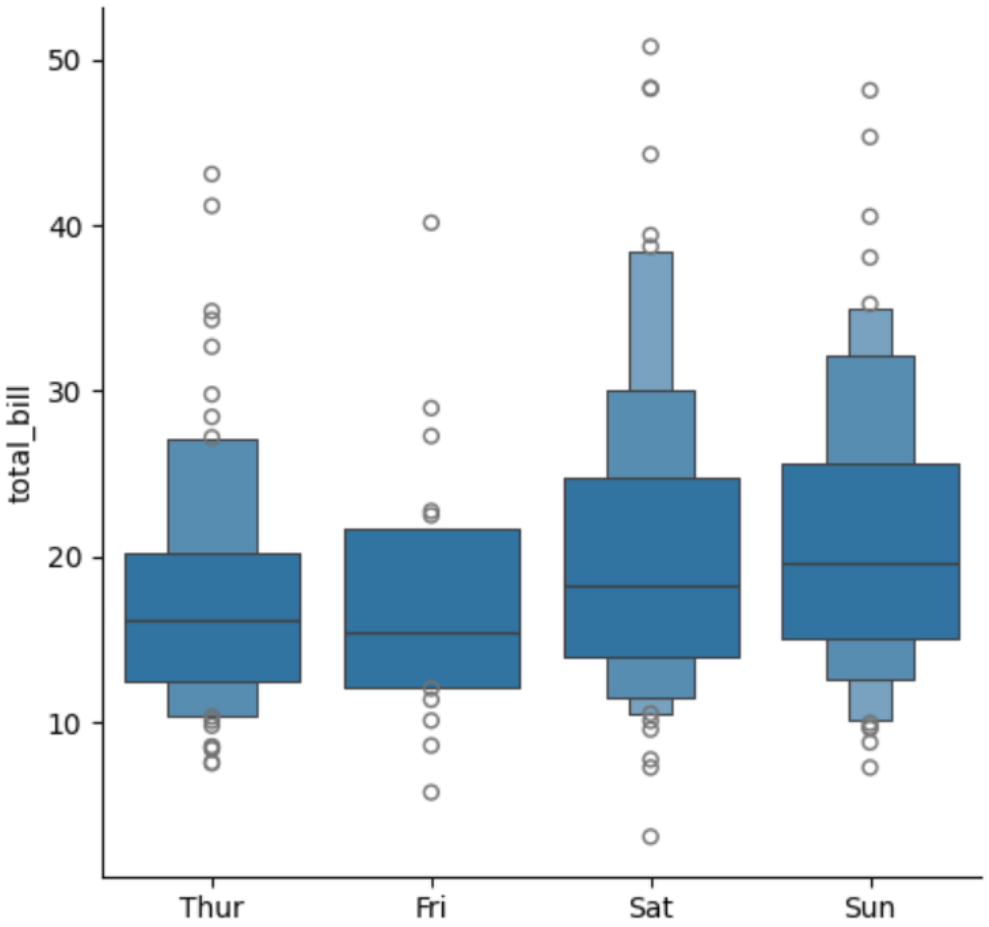
When adding a **hue** semantic, the box for each level of the semantic variable is made narrower and shifted along the categorical axis



Example – boxenplot()

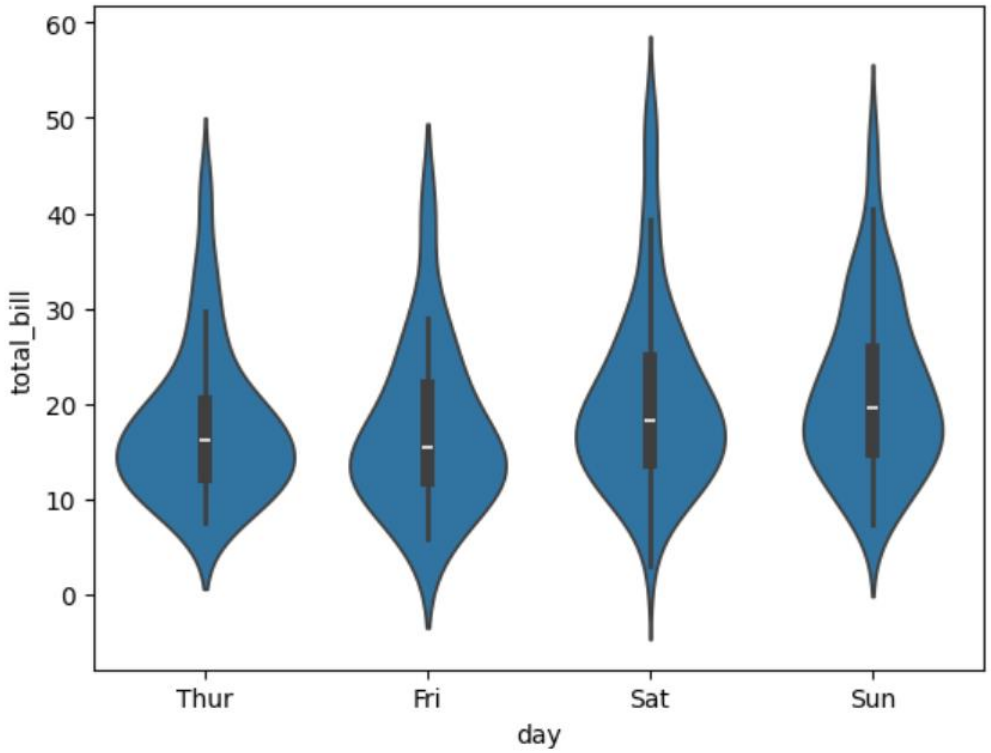
VIGNAN'S
FOUNDATION FOR SCIENCE, TECHNOLOGY & RESEARCH
(Deemed to be University) - Estd. u/s 3 of UGC Act 1956

```
sns.catplot(data=tips, x="day", y="total_bill", kind="boxen")
```

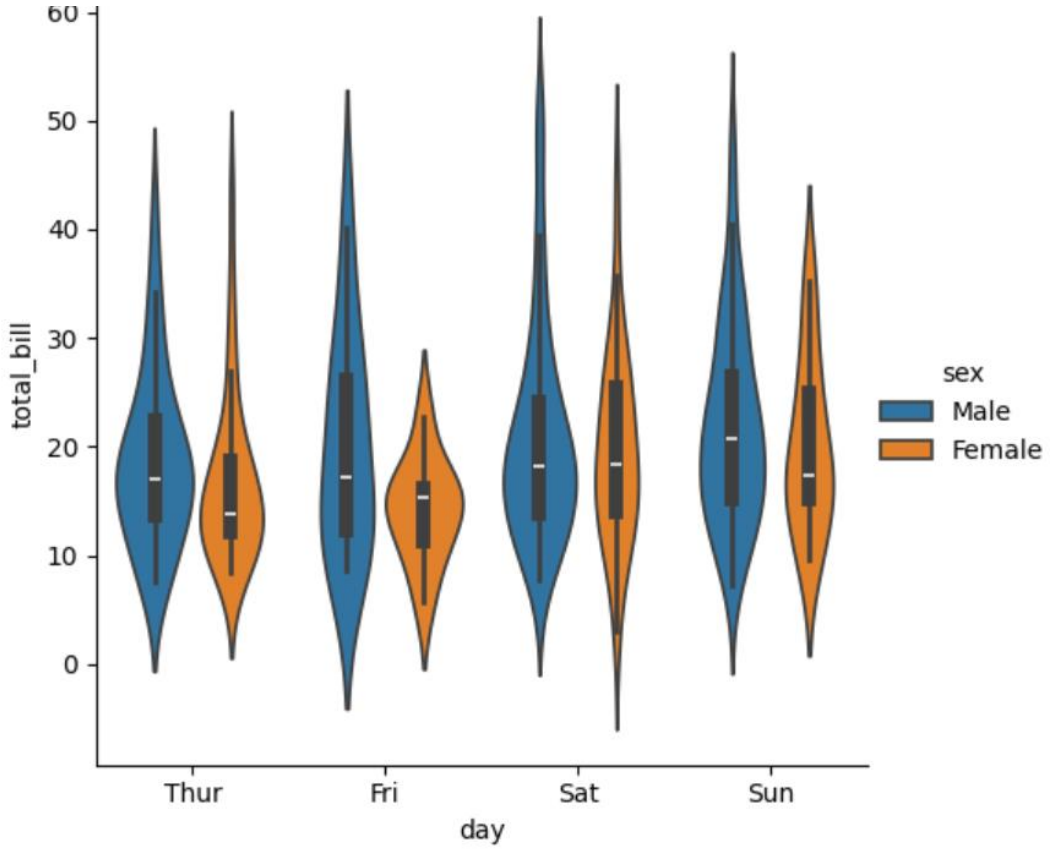


Example – violinplot()


```
sns.catplot(data=tips,
        x="day",
        y="total_bill“,
        kind="violin")
```




```
sns.catplot(
    data=tips, x="day", y="total_bill",
hue="sex", kind="violin",
)
```



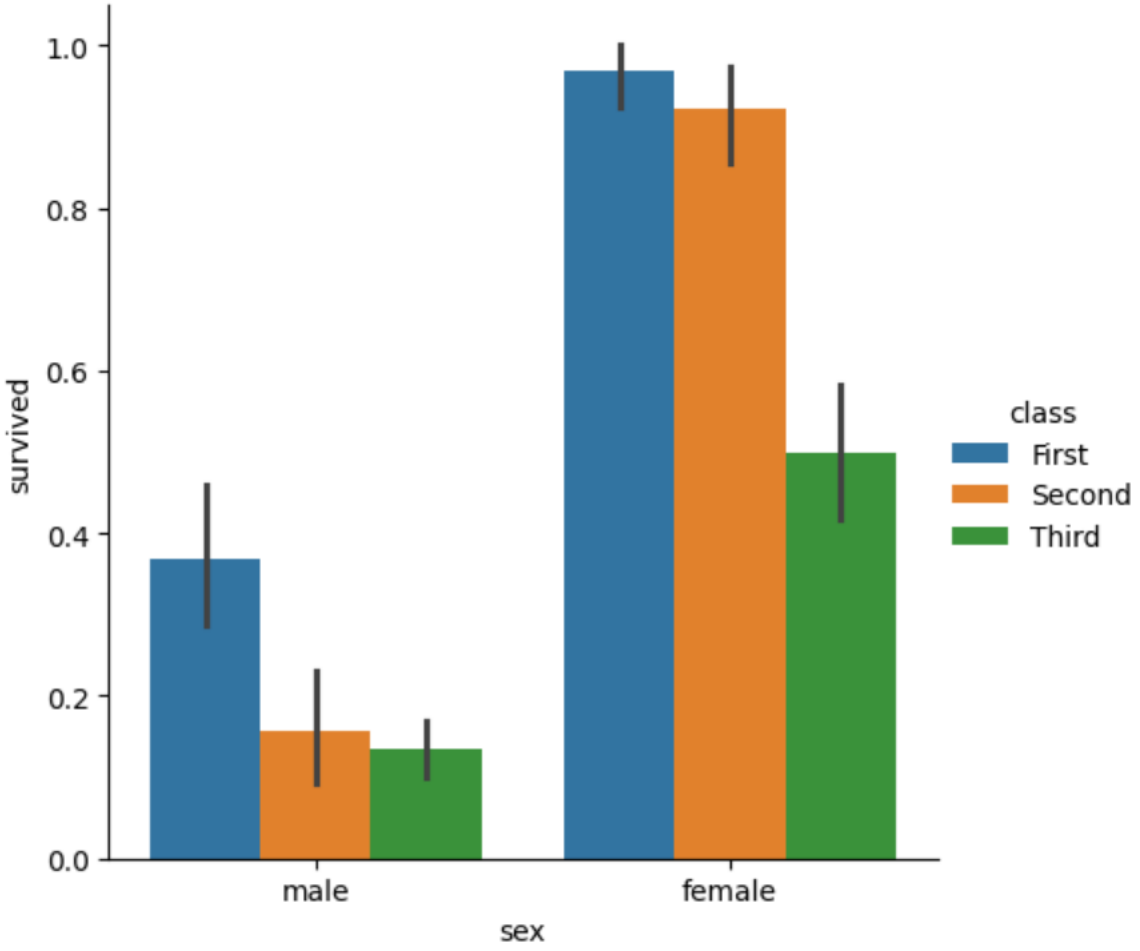
Categorical estimate plots

- Sometimes, do not need to display every individual data point.
- Instead, summarize the dataset by showing: the average value or typical behavior of the data
- **Bar plot**
  - display average values for categories.
  - The bar height represents the **mean (average) value** by default.
- **Point plot**
  - Instead of displaying full bars like a bar plot, a point plot shows **mean** values with **confidence** intervals.

Example – barplot()

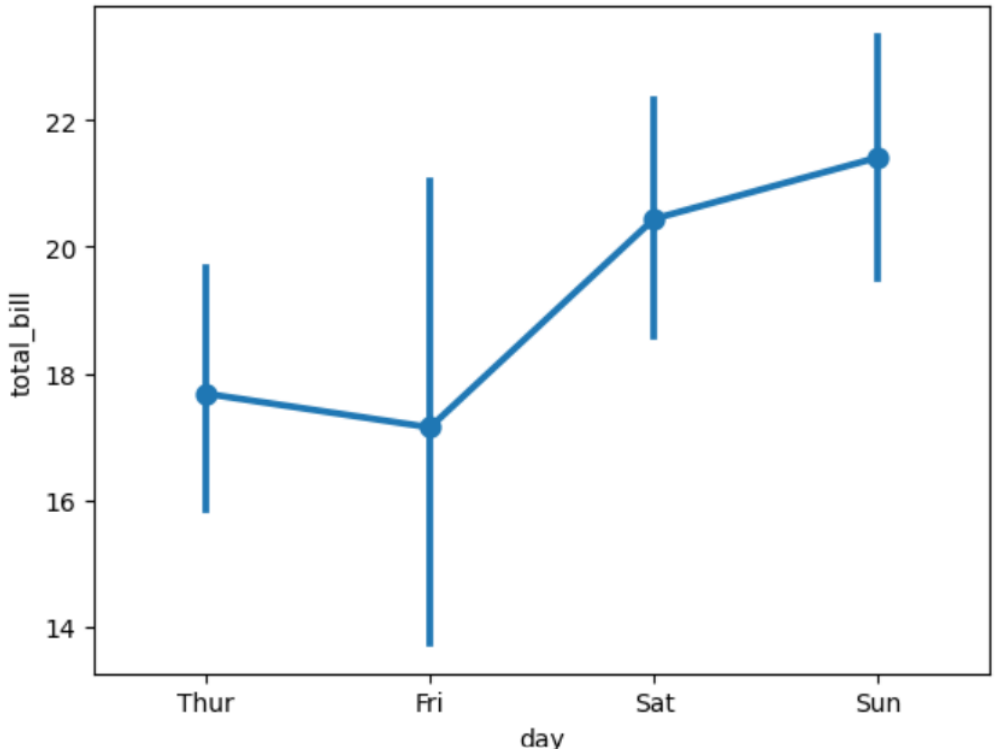

```
titanic = sns.load_dataset("titanic")
sns.catplot(data=titanic, x="sex",
        y="survived",
        hue="class",
        kind="bar")
```

- **Bar height** = average value
- **Black line** = uncertainty or confidence interval of that average

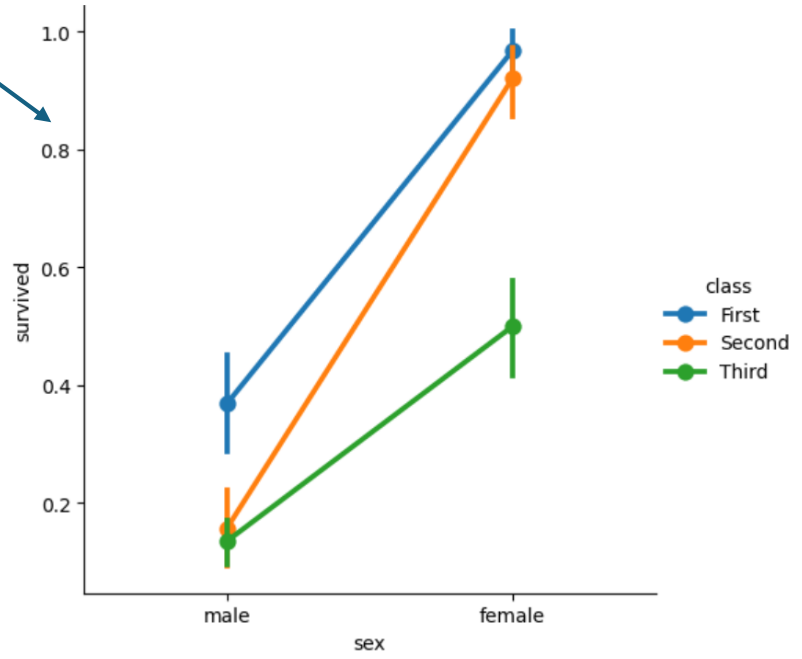


Example – pointplot()


```
sns.catplot(data=tips, x="day", y="total_bill", kind="point")
```




```
titanic = sns.load_dataset("titanic")
sns.catplot(data=titanic, x="sex",
            y="survived",
            hue="class",
            kind="point")
```



Relational plots

- Seaborn provides several powerful relational visualization tools:
  - Heat map
  - Pair Plot
  - Joint Plot
  - Facet Grid

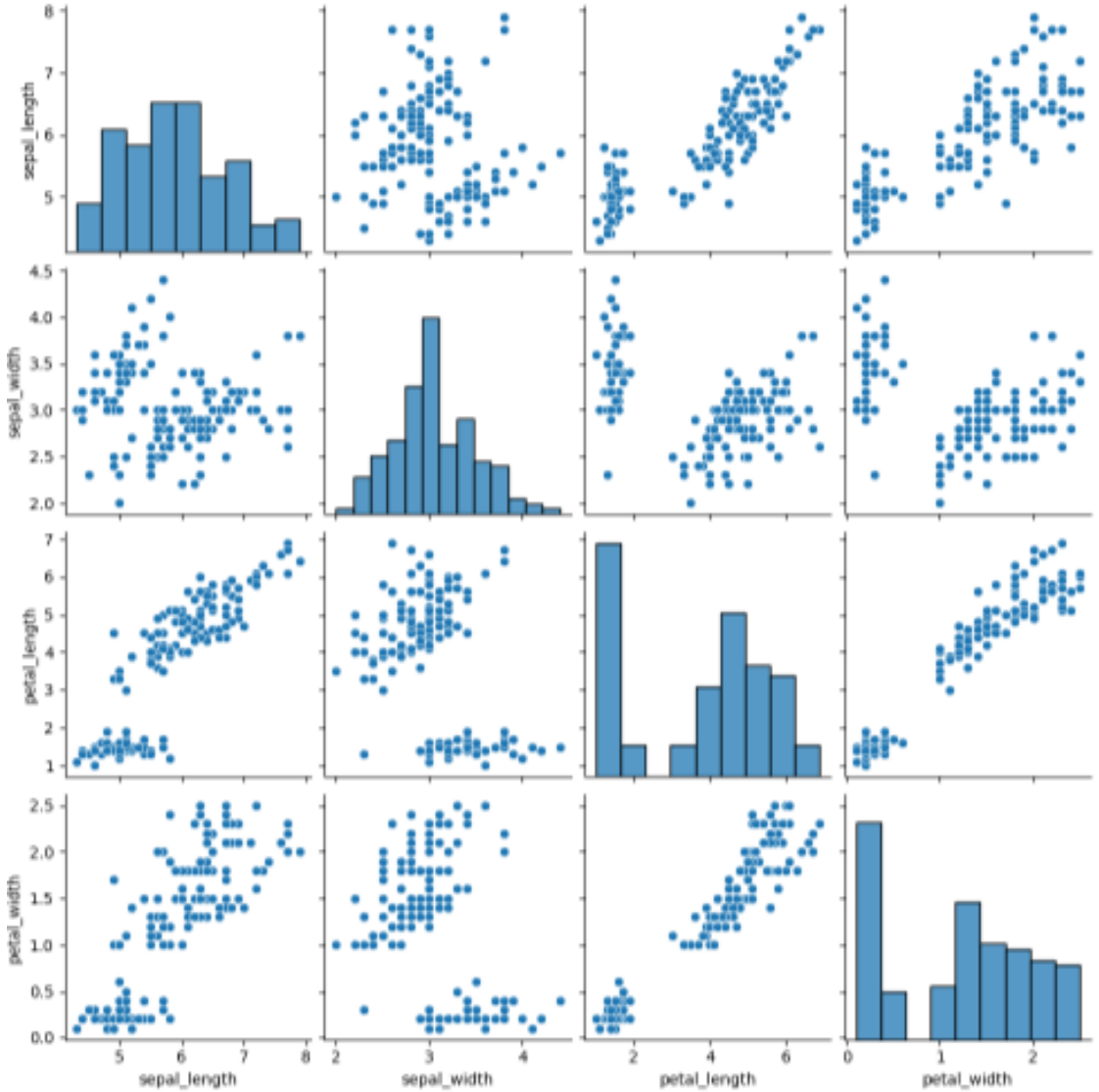
- **Heat maps**
  - It is a graphical representation of data where values are represented using **color intensity**.
  - These are commonly used to visualize: Correlation matrices, Numerical data tables, Matrix-based datasets

Pair Plot

- Plot pairwise relationships between numerical features in a dataset.
- Displays:
  - **Off-diagonal plots** → Scatter plots (relationship between two variables)
  - **Diagonal plots** → Histograms or KDE plots (distribution of each variable)
- It is very useful for **multivariate data analysis**.

Pair Plot

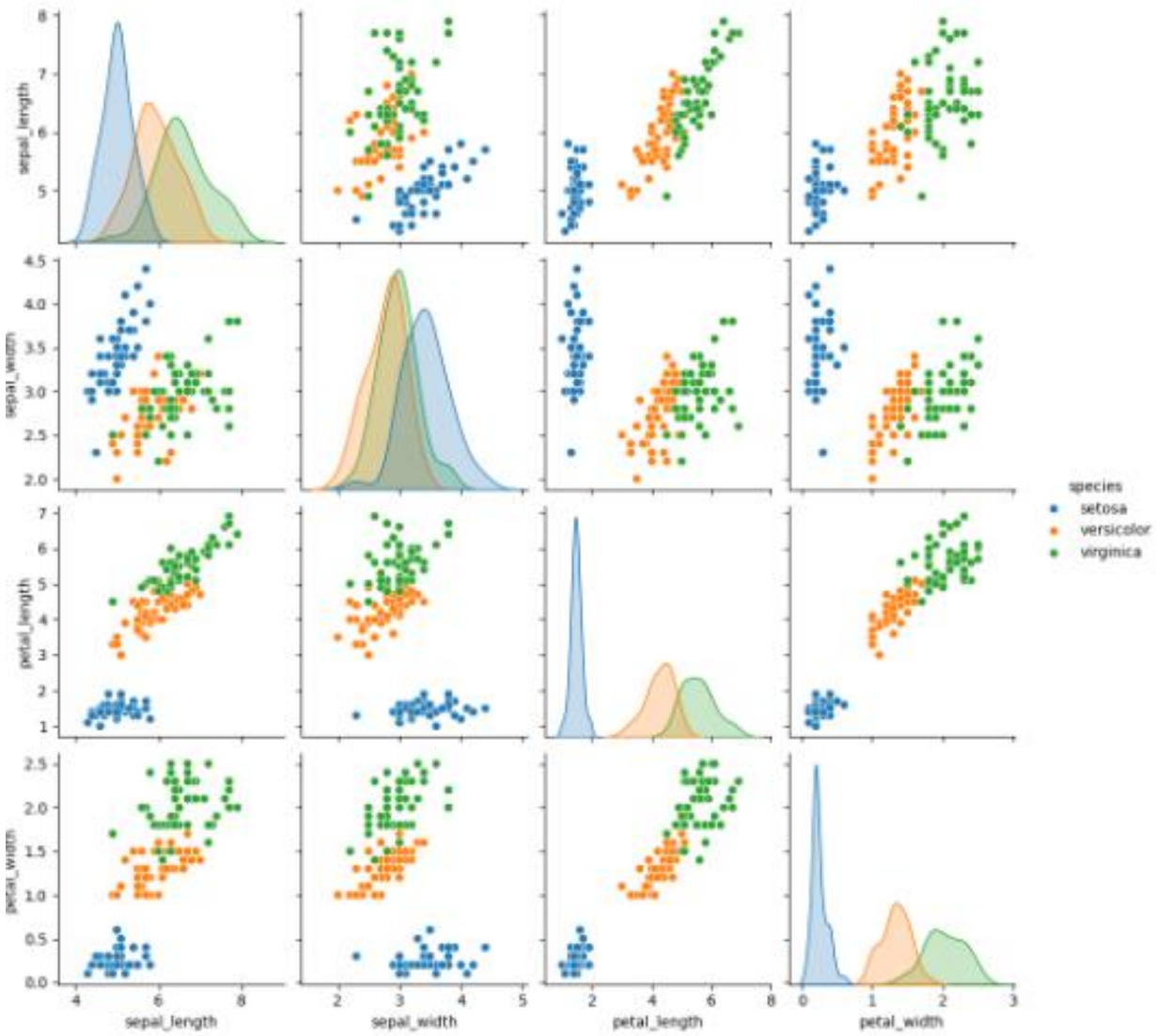

```
iris = sns.load_dataset("iris")
sns.pairplot(iris)
```



https://seaborn.pydata.org/generated/seaborn.pairplot.html#seaborn.pairplot

Pair Plot


```
iris = sns.load_dataset("iris")
sns.pairplot(iris, hue="species")
```

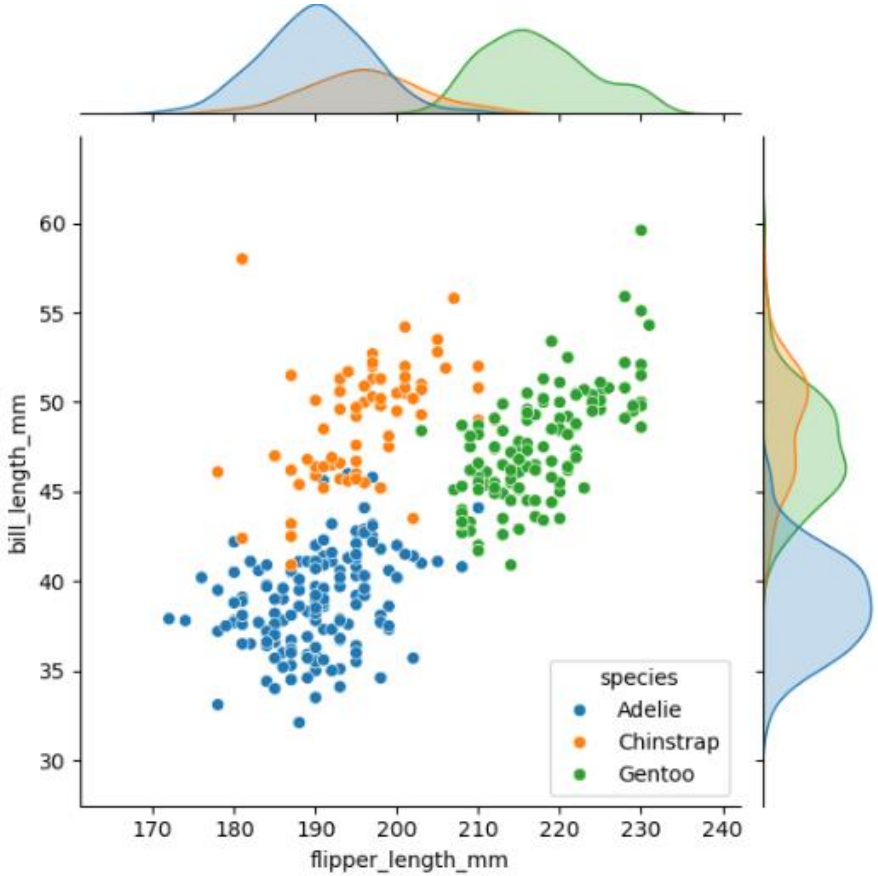


Joint Plot

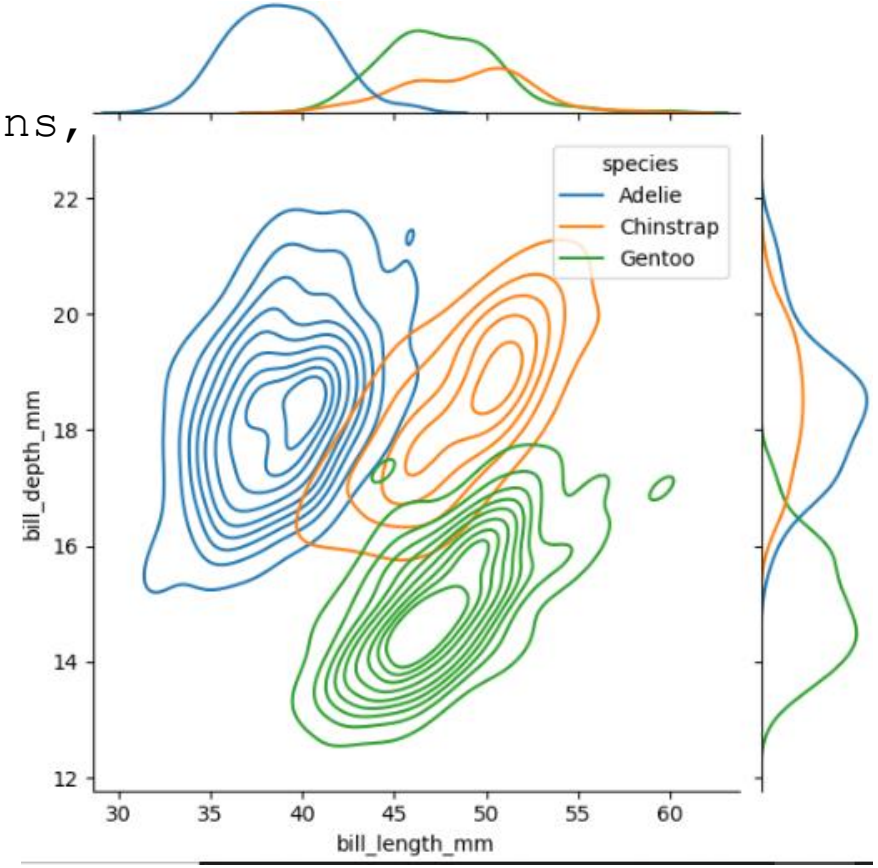
- `jointplot` is used to visualize:
  - The relationship (joint distribution) between two variables
  - Adding marginal axes that show the univariate distribution of each one separately
- A joint plot has three parts:
  - Center Plot – Shows relationship (e.g., scatter plot)
  - Top Margin – Distribution of X variable
  - Right Margin – Distribution of Y variable

Joint Plot


```
penguins = sns.load_dataset("penguins")
sns.jointplot(data=penguins, x="flipper_length_mm", y="bill_length_mm", hue="species")
```




```
sns.jointplot(data=penguins,
    x="bill_length_mm",
    y="bill_depth_mm",
    hue="species",
    kind="kde")
```



Facet Grid

- FacetGrid is a Seaborn class used to create multiple plots based on subsets of a dataset.
- Each subset of the dataset is displayed in a separate subplot (facet).
- It helps visualize:
  - Distribution of a variable
  - Relationships between variables
  - Comparisons across categories
- A FacetGrid can display data in three dimensions: row, col, and hue
  - Row and Column determine the grid layout
  - Hue adds another grouping variable using colors.

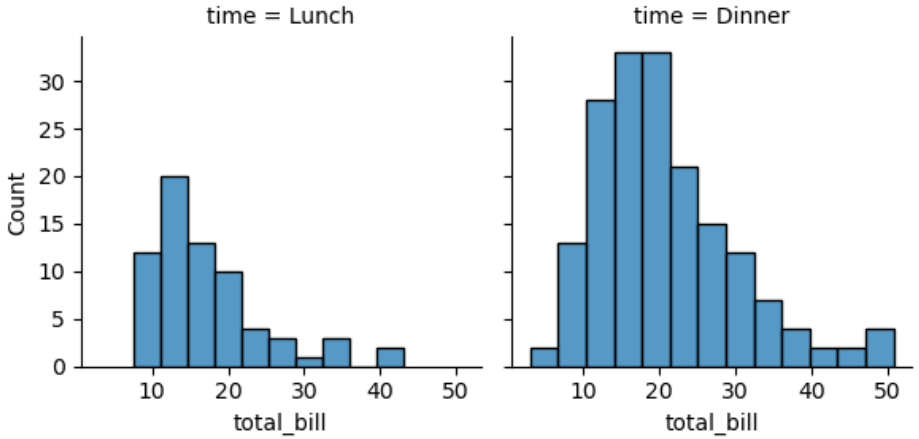
Facet Grid

- Several Seaborn plotting functions internally use FacetGrid: relplot(), displot(), catplot(), and lmplot()
- These functions create a FacetGrid automatically and return the object for further customization.
- To use FacetGrid, we first initialize the object with:
  - A DataFrame
  - Variables for row, column, or hue
- These variables should be categorical or discrete

Facet Grid

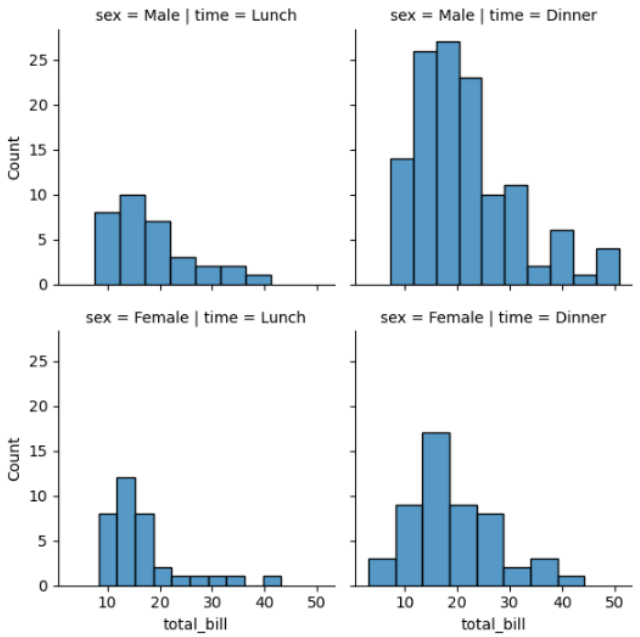

```
tips = sns.load_dataset("tips")

g = sns.FacetGrid(tips, col="time")
g.map(sns.histplot, "total_bill")
```



- `col="time"` → Creates separate plots for Lunch and Dinner
- `map()` → Applies a plotting function to each subset

```
g = sns.FacetGrid(tips, col="time",  row="sex")
g.map_dataframe(sns.histplot, x="total_bill")
```



https://seaborn.pydata.org/generated/seaborn.FacetGrid.html

Reference text books:

- Wes McKinney. Python for Data Analysis: Data Wrangling with pandas, NumPy and Jupyter. O'Reilly, 2022, 3rd Edition
- Kennedy R. Behrman, “Foundational Python for Data Science”, Pearson Education, Inc. 2022.